

AWS Core Services — Foundations Study Guide

Focus Area: AWS Compute, Storage & Networking Fundamentals **Objective:** Understand and hands-on configure EC2, Security Groups, AMIs, EBS, Snapshots, and Network Interfaces from scratch.

Table of Contents

1. [EC2 — Elastic Compute Cloud](#)
2. [Security Groups](#)
3. [AMI — Amazon Machine Image](#)
4. [EBS — Elastic Block Store](#)
5. [EBS Snapshots](#)
6. [Network Interfaces & IP Addressing](#)

1. EC2 — Elastic Compute Cloud

What is EC2?

Amazon EC2 (Elastic Compute Cloud) is a service that lets you rent virtual servers in the cloud on demand. Instead of buying physical hardware, you spin up a virtual machine in minutes, configure it exactly how you want, and pay only for the time you actually use it.

A practical way to picture it — imagine you need a powerful computer for a project. Instead of spending thousands buying one, you rent one from AWS, use it for a few hours or months, and stop paying the moment you're done.

EC2 handles the physical hardware, data center, and power supply. You handle the operating system, software, and what runs on it.

Traditional Setup vs EC2:

Physical Server:

Buy hardware upfront →
Wait weeks to deploy →
Fixed capacity →
You maintain hardware →
High upfront cost →

EC2 Instance:

Launch in 60 seconds
Pay per hour/second
Scale up or down anytime
AWS maintains hardware
Zero upfront cost

Core Components

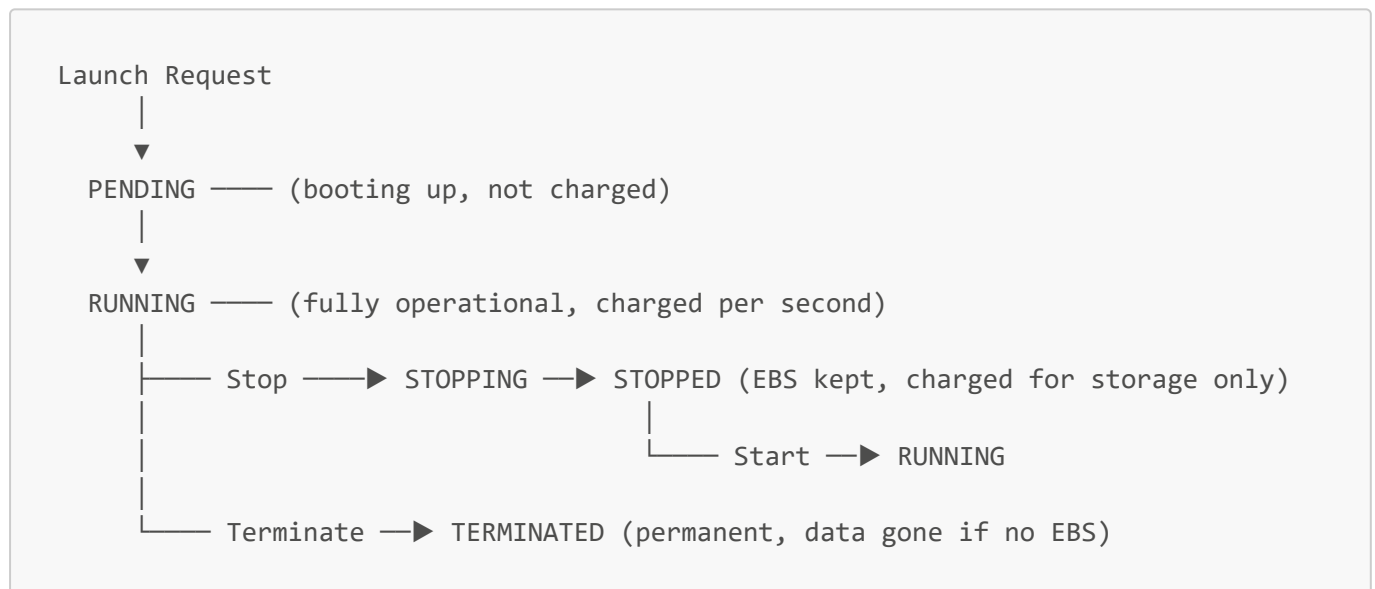
Instance Families

AWS groups instance types into families based on what they're optimized for:

Family	Optimized For	Common Types	Example Workload
General Purpose	Balanced CPU + RAM	t2.micro, t3.small, m5.large	Web servers, dev environments
Compute Optimized	High CPU performance	c5.xlarge, c6g.large	Gaming servers, HPC, video encoding
Memory Optimized	Large RAM capacity	r5.large, x1e.xlarge	In-memory databases, real-time analytics
Storage Optimized	High disk I/O	i3.large, d3.xlarge	Data warehouses, Hadoop clusters
Accelerated	GPU / FPGA	p3.xlarge, g4dn.large	Machine learning, 3D rendering

t2.micro is free tier eligible — perfect for learning and small projects.

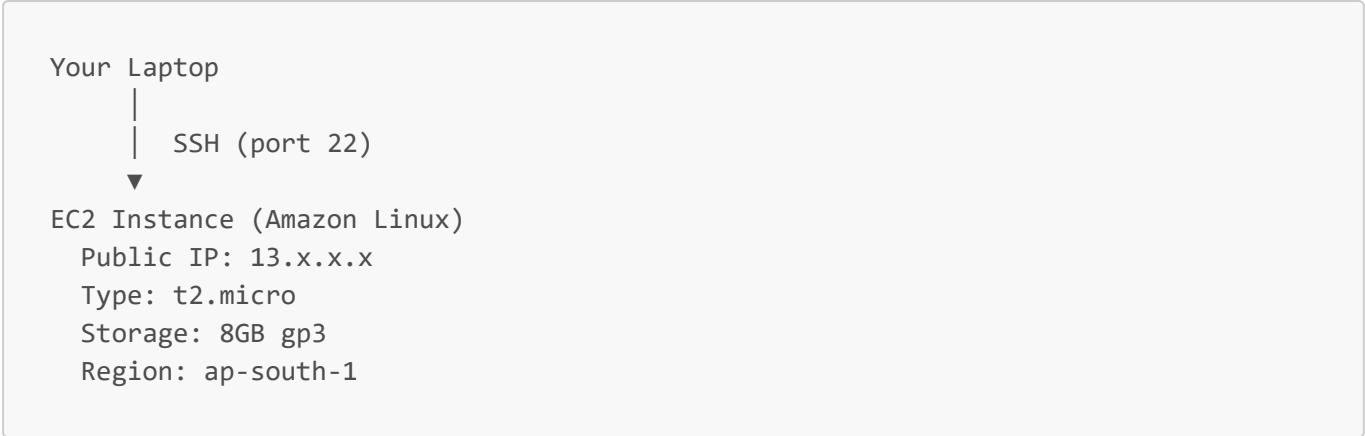
Instance Lifecycle States



State	What's Happening	Are You Charged?
 Pending	Instance is starting up	No
 Running	Fully operational	Yes — per second
 Stopping	Shutting down in progress	No
 Stopped	Off — EBS volumes retained	EBS storage cost only
 Terminated	Permanently deleted	No

Hands-On: Launch Your First EC2 Instance

What We're Building



Step-by-Step

1. Open EC2 Dashboard

- AWS Console → Search "EC2" → Click EC2 Dashboard
- Click the orange "**Launch Instance**" button

2. Configure Instance Details

Field	Value
Name	MyFirstWebServer
AMI	Amazon Linux 2023 (Free Tier Eligible)
Instance Type	t2.micro (Free Tier Eligible)
Key Pair	Create new → Name: my-keypair → Download .pem file

⚠ The **.pem** file is your only way to SSH in. Save it somewhere safe — AWS will never show it again.

3. Network & Security Settings

Field	Value
VPC	Default VPC
Subnet	Any available subnet
Auto-assign Public IP	Enable
Security Group	Create new → Allow SSH (22), HTTP (80), HTTPS (443)

4. Storage

Field	Value
Root Volume	8 GB gp3 (default)
Delete on termination	Yes (default)

5. Launch and Connect

```
# Fix key permissions (required on Linux/Mac)
chmod 400 my-keypair.pem

# SSH into instance
ssh -i my-keypair.pem ec2-user@YOUR_PUBLIC_IP

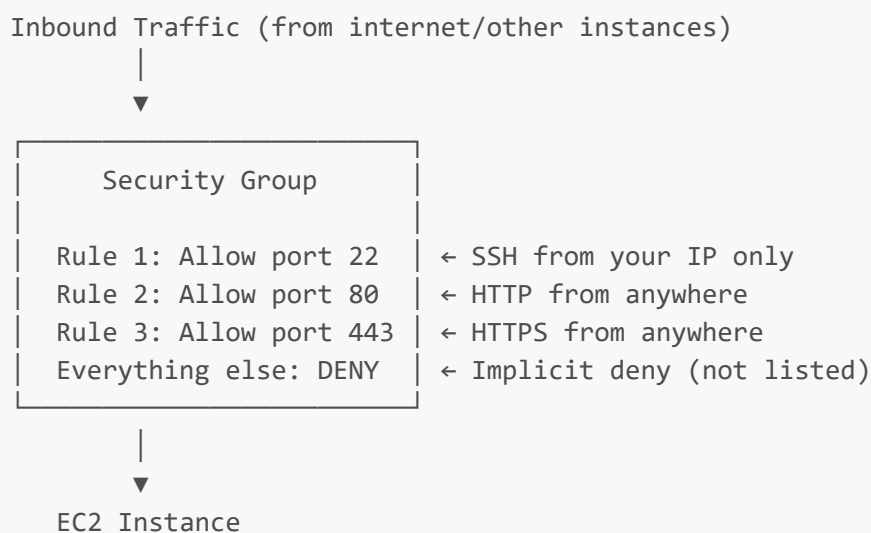
# Verify you're inside the instance
whoami
# Output: ec2-user
```

2. Security Groups

What is a Security Group?

A **Security Group** is a set of firewall rules that controls what network traffic is allowed to reach your EC2 instance and what traffic can leave it. Every EC2 instance must have at least one Security Group — there's no way around it.

Think of a Security Group like a bouncer at a nightclub. The bouncer has a list of who gets in (inbound rules) and checks people as they arrive. If you're not on the list, you're not getting in — regardless of how you got to the door.



How Security Groups Work

Core Rules:

Default Behavior:

Inbound → ALL DENIED by default (you open only what you need)
Outbound → ALL ALLOWED by default (instances can reach internet)

Key Properties:

- ✓ Stateful – allow inbound port 80, response automatically allowed
- ✓ Allow-only – no DENY rules exist, only ALLOW rules
- ✓ Multi-attach – one instance can have multiple security groups
- ✓ Instant effect – rule changes apply without restarting instance
- ✓ SG referencing – use another SG as source instead of an IP address

Stateful Explained:

Without stateful (how NACLs work):

You open port 80 inbound ✓

You must also open ephemeral ports outbound for response ⚠

With stateful (how Security Groups work):

You open port 80 inbound ✓

Response traffic automatically allowed ✓ (no extra rule needed)

Rule Fields Explained

Field	What It Means	Example Value
Type	Predefined protocol shortcut	SSH, HTTP, MySQL/Aurora
Protocol	Network protocol	TCP, UDP, ICMP
Port Range	Which port(s) to open	22, 80, 443, 3306, 0-65535
Source (Inbound)	Who is allowed to connect	0.0.0.0/0, My IP, another SG
Destination (Outbound)	Where traffic can go	0.0.0.0/0

Source Options Explained:

- 0.0.0.0/0 → Anyone on the internet (use carefully)
- My IP → Only your current IP address
- 10.0.0.0/16 → Any IP within your VPC
- sg-xxxxxxx → Only instances that have this specific Security Group

Hands-On: Create and Attach a Security Group

Step-by-Step

1. Create the Security Group

- EC2 Dashboard → Left sidebar → **Security Groups**

- Click "**Create security group**"

Field	Value
Name	WebServer-SG
Description	Allow HTTP, HTTPS from everywhere, SSH from my IP only
VPC	Select your VPC

2. Add Inbound Rules

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	My IP	Admin access — your IP only
HTTP	TCP	80	0.0.0.0/0	Web traffic from anyone
HTTPS	TCP	443	0.0.0.0/0	Secure web traffic from anyone

3. Outbound Rules

- Leave default: All traffic → 0.0.0.0/0
- This lets your instance download updates, call APIs, etc.

4. Attach to an Existing Instance

```
EC2 → Instances → Select your instance
→ Actions → Security → Change security groups
→ Search and select "WebServer-SG"
→ Click "Save"
```

Changes apply instantly — no reboot needed.

3. AMI — Amazon Machine Image

What is an AMI?

An **Amazon Machine Image (AMI)** is a saved template of an EC2 instance — essentially a complete snapshot of everything needed to recreate that exact server from scratch. When you launch an EC2 instance, you're always launching from an AMI.

Real-world analogy: Imagine you spent a week setting up a perfect workstation — installed all your tools, configured everything exactly right, personalized every setting. An AMI is like cloning that entire workstation so you (or your team) can spin up identical copies instantly, anytime, without repeating all that setup work.

One AMI → Launch 1 instance (single server)
One AMI → Launch 10 instances (instant fleet)
One AMI → Launch in any region (global deployment)

What an AMI Contains

AMI Contents:

- Root Volume Snapshot
 - └ Operating system (Linux, Windows)
 - └ All installed software (NGINX, Node.js, MySQL)
 - └ All your custom configurations
- 🔒 Launch Permissions
 - └ Which AWS accounts can use this AMI
- 💾 Block Device Mapping
 - └ Which EBS volumes to attach on launch
 - └ Volume sizes and types
- ⚙️ Metadata
 - └ Architecture: x86_64 or ARM
 - └ Virtualization type: HVM
 - └ Region where AMI lives

AMI Types

Category	Created By	Best For	Example
• AWS Provided	Amazon	Starting fresh, official base OS	Amazon Linux 2023, Ubuntu 22.04
 AWS Marketplace	Third-party vendors	Pre-packaged software stacks	Bitnami WordPress, Palo Alto Firewall
 Community AMIs	Public AWS users	Specialized or uncommon setups	Custom distros, research images
 Your Custom AMIs	You	Pre-configured app environments	Your app server fully configured

AMI Regions and Sharing

AMI Scope Rules:

- ✗ AMI in ap-south-1 CANNOT directly launch in us-east-1
- ✓ Use "Copy AMI" to replicate it to another region

Sharing Options:

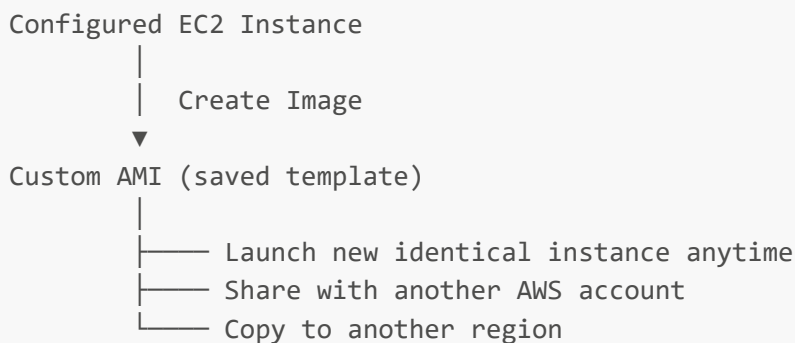
- 🔒 Private (default) → Only your AWS account
- 👤 Specific accounts → Share with exact AWS Account IDs
- 🌐 Public → Anyone in the world can use it

When to copy AMIs across regions:

- Setting up disaster recovery in a secondary region
- Deploying identical servers globally for lower latency
- Sharing a pre-built environment with a team in another region

🚀 Hands-On: Create a Custom AMI and Share It

What We're Doing



Step-by-Step

1. Prepare Your Instance

Make sure everything is installed and configured:

```
# Example: install and verify NGINX
sudo yum update -y
sudo yum install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
nginx -v
```

2. Create the AMI

- EC2 → Instances → Select your configured instance
- **Actions → Image and Templates → Create Image**

Field	Value
Image Name	MyApp-AMI-v1
Description	NGINX web server — configured July 2025
No Reboot	Leave unchecked (checked = risky, may cause inconsistency)

- Click **"Create Image"**
- Navigate to **EC2** → **AMIs** — status shows **Pending** → wait for **Available** (3–10 min)

3. Share the AMI

- Select your AMI → **Actions** → **Edit AMI Permissions**
- Change from Public to Private
- Under **"Add AWS Account ID"** → enter the target account's 12-digit ID
- Click **"Add permission"** → **"Save changes"**

4. Copy AMI to Another Region

- Select AMI → **Actions** → **Copy AMI**
- Select destination region (e.g., **us-east-1**)
- Click **"Copy AMI"**
- Switch to that region and check under AMIs — it will appear there



4. EBS — Elastic Block Store

💡 What is EBS?

Amazon Elastic Block Store (EBS) provides persistent storage volumes that you attach to EC2 instances. Think of EBS volumes exactly like hard drives or SSDs — they store data, survive reboots, and can be detached and reattached to different instances.

The most important thing about EBS: **data persists independently from the instance lifecycle**. If you stop your EC2 instance and restart it tomorrow, all your EBS data is still there exactly as you left it. This is completely different from instance store (ephemeral storage), which vanishes the moment an instance stops.

EBS vs Instance Store:

EBS Volume:

Instance stops → Data SAFE ✓

Instance starts → Data still there ✓

Instance terminates → Data SAFE (if "delete on termination" = No) ✓

Instance Store (ephemeral):

Instance stops → Data GONE ✗

Instance terminates → Data GONE ✗

Good only for temporary scratch data

EBS Volume Types

Type	Category	Max IOPS	Max Throughput	Best For
gp3	SSD	16,000	1,000 MB/s	Default choice — OS volumes, apps, dev
gp2	SSD	16,000	250 MB/s	Legacy general purpose (prefer gp3)
io2 Block Express	SSD	256,000	4,000 MB/s	Critical databases — Oracle, SAP HANA
io1	SSD	64,000	1,000 MB/s	High I/O apps needing guaranteed performance
st1	HDD	500 MB/s throughput	500 MB/s	Big data, log processing, Kafka
sc1	HDD	250 MB/s throughput	250 MB/s	Cold archives, rarely accessed backups

Rule of thumb: Use **gp3** for almost everything. Switch to **io2** only when your database demands guaranteed ultra-high IOPS.

Important EBS Characteristics

-  **AZ-Locked:**
EBS volume in ap-south-1a → can ONLY attach to EC2 in ap-south-1a
To move to 1b → take snapshot → create new volume in 1b from snapshot
-  **One-to-One Attachment:**
One EBS volume → One EC2 instance at a time
Exception: io1/io2 support Multi-Attach (advanced use case)
-  **Elastic Sizing:**
Increase size → No downtime needed
Change type → No downtime needed (e.g., gp2 → gp3)
Decrease size → NOT supported (can only grow, not shrink)
-  **Encryption:**
Uses AES-256 encryption
Encrypting a volume also encrypts all snapshots made from it
Encrypted volumes → encrypted AMIs → encrypted new volumes

Hands-On: Create, Attach, and Mount an EBS Volume

What We're Building

```
EC2 Instance
├─ /dev/xvda → Root volume (OS) – 8GB gp3
└─ /dev/xvdf → New data volume – 20GB gp3 ← We're adding this
    Mounted at: /data
```

Step-by-Step

1. Create the EBS Volume

- EC2 Dashboard → Elastic Block Store → **Volumes** → **Create volume**

Setting	Value
Volume Type	gp3
Size	20 GiB
Availability Zone	⚠️ MUST match your EC2 instance's AZ (e.g., ap-south-1a)
IOPS	3000 (default gp3)
Throughput	125 MB/s (default)
Tag Name	DataVolume-01

- Click **"Create volume"** → Status shows **Available**

2. Attach the Volume to Your Instance

- Select the volume → **Actions** → **Attach volume**
- Select your EC2 instance from dropdown
- Device name: **/dev/xvdf**
- Click **"Attach volume"** → Status changes to **In-use**

3. Format and Mount Inside EC2

```
# SSH into your instance first
ssh -i my-keypair.pem ec2-user@YOUR_PUBLIC_IP

# Verify the new disk is visible
sudo lsblk
# You should see xvdf listed

# Format the volume (only do this on a NEW, empty volume)
sudo mkfs -t ext4 /dev/xvdf

# Create a directory to mount it
sudo mkdir /data

# Mount the volume
sudo mount /dev/xvdf /data
```

```
# Verify it's mounted
df -h | grep /data
# Should show 20G available

# Make it auto-mount after every reboot
echo '/dev/xvdf /data ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab

# Test auto-mount config
sudo mount -a
```

⚠️ Only run `mkfs` on a brand new empty volume. Running it on a volume with data will erase everything.

📷 5. EBS Snapshots

💡 What is an EBS Snapshot?

An **EBS Snapshot** is a point-in-time backup of an EBS volume stored durably in Amazon S3 (managed behind the scenes by AWS — you won't see it as an S3 bucket). It captures the exact byte-for-byte state of your volume at the moment you triggered the snapshot.

Practical example: Your EC2 instance runs a production database. Every night at midnight, you take a snapshot. If something goes catastrophically wrong on Tuesday afternoon, you can restore from Monday night's snapshot and only lose one day's data — rather than everything.

EBS Volume Timeline:

```
Monday midnight → Snapshot 1 taken ✅
Tuesday 2pm      → Ransomware attack 🧑‍🔪
Tuesday 2:01pm  → Restore from Snapshot 1 ✅
Result: Only lost ~14 hours of data, not everything
```

🔗 How Incremental Snapshots Work

After the first full snapshot, AWS only saves what changed — not the entire volume each time.

Volume Size: 100 GB

Snapshot 1 (First ever):

└─ Copies ALL 100GB → Stored: 100 GB

Snapshot 2 (Next day, 15GB changed):

└─ Copies only 15GB changed blocks → Stored: 15 GB

Snapshot 3 (Day after, 8GB changed):

└─ Copies only 8GB changed blocks → Stored: 8 GB

Total storage used: $100 + 15 + 8 = 123$ GB
Without incremental: $100 + 100 + 100 = 300$ GB
Savings: 177 GB saved! 💰

Deleting an older snapshot does NOT break newer ones. AWS reorganizes the data blocks automatically.

🔧 What You Can Do with Snapshots

EBS Snapshot

- 🔄 Restore → Create new EBS volume from snapshot
(recover data, move to different AZ)
- 🌐 Copy → Send to another AWS region
(disaster recovery, geographic redundancy)
- 👤 Share → Grant access to specific AWS accounts
(hand off data to partners/teams)
- 🖼️ AMI → Create AMI from a root volume snapshot
(package an OS + software image)
- ⌚ Automate → Use Data Lifecycle Manager (DLM)
(scheduled automatic snapshots, no manual work)

🚀 Hands-On: Create, Restore, and Copy Snapshots

Step-by-Step

1. Create a Snapshot

- EC2 → Elastic Block Store → **Volumes**
- Select the volume you want to back up
- Actions** → **Create snapshot**

Field	Value
Description	Pre-upgrade backup — July 2025
Tag Name	Pre-Upgrade-Snap-01

- Click "**Create snapshot**"
- Navigate to **EC2** → **Snapshots** → watch status: Pending → Completed

2. Restore from Snapshot (Create New Volume)

- Select the snapshot → **Actions** → **Create volume from snapshot**

Setting	Value
Volume Type	gp3
Size	Same or larger than original
Availability Zone	Target AZ for the new volume

- Click "**Create volume**" → attach to an instance as needed

3. Copy Snapshot to Another Region

- Select snapshot → **Actions** → **Copy snapshot**
- Select destination region (e.g., **us-east-1**)
- Click "**Copy snapshot**" → switch to that region to monitor

4. Share Snapshot with Another Account

- Select snapshot → **Actions** → **Modify permissions**
- Add the target AWS Account ID
- Click "**Save changes**"
- The other account finds it under **Private snapshots** in their console

6. Network Interfaces & IP Addressing

What is an ENI?

An **Elastic Network Interface (ENI)** is a virtual network card that attaches to an EC2 instance. Just like a physical server needs a network card to connect to a network, an EC2 instance needs an ENI to connect to a VPC.

Every EC2 instance gets one ENI automatically on launch — called the **primary network interface**. You can attach additional ENIs for advanced setups like dual-homed instances (connected to two subnets simultaneously) or failover configurations.

```

EC2 Instance
├── Primary ENI (eth0)    ← Always present, created automatically
│   ├── Private IP: 10.0.1.25
│   ├── Public IP: 13.x.x.x (if in public subnet)
│   └── Security Groups: [WebServer-SG]
└── Secondary ENI (eth1) ← Optional, you attach manually
    ├── Private IP: 10.0.2.30
    └── Security Groups: [App-SG]
```

ENI Failover Use Case:

Server A fails:

ENI (with static private IP) detaches from Server A

↓

ENI reattaches to Server B

↓

Server B now has the same private IP as Server A had

↓

All internal traffic continues without IP changes ✓

The Three Types of IP Addresses

Your EC2 Instance has up to 3 types of IPs:

EC2 Instance

- Private IP: 10.0.1.25
Always present, internal VPC only
- Public IP: 54.12.34.56
Auto-assigned, changes on stop/start
- Elastic IP: 13.110.0.5
Static, you own it, never changes

Private IP

Every EC2 instance in a VPC is assigned a private IP from the VPC's CIDR range automatically. This IP is used for all traffic that stays inside AWS — instance-to-instance communication, talking to RDS databases, connecting to load balancers, etc.

Private IP Properties:

Assigned from VPC CIDR (e.g., 10.0.0.0/16)

Stays the same when instance is stopped and restarted ✓

NOT reachable from the public internet ✗

Used for all internal VPC communication

AWS provides internal DNS: ip-10-0-1-25.ec2.internal

Can assign multiple private IPs to one ENI

Public IP

Automatically assigned when an EC2 instance launches in a public subnet (if auto-assign is enabled). Allows direct internet access through the Internet Gateway.

⚠ Critical Behavior:

Instance Running → Public IP: 54.12.34.56
Instance STOPPED → Public IP: GONE ❌
Instance STARTED → Public IP: 54.98.11.200 (completely different!)

This means:

- DNS records pointing to this IP break
- Hardcoded IPs in configs break
- Use Elastic IP if you need a stable address

● Elastic IP (EIP)

An **Elastic IP** is a static public IPv4 address that you reserve from AWS and own until you explicitly release it. Unlike the auto-assigned public IP, an Elastic IP stays yours regardless of what happens to the underlying instance.

Elastic IP Benefits:

- ✅ Never changes – stop, start, even replace instance
- ✅ Can be moved between instances instantly
- ✅ Great for production servers that need a fixed IP
- ✅ Works for DNS A records (IP stays the same forever)
- ✅ Useful for whitelisting (partners allow your fixed IP)

Elastic IP Cost Warning:

- 💰 FREE when attached to a RUNNING instance
- 💰 \$0.005/hour when NOT attached (or instance stopped)

Always release EIPs you're not using!
5 EIPs per region per account (default limit)

Elastic IP Failover Pattern:

Normal: Server A ← Elastic IP (3.110.x.x)

Server A fails: Server A ❌

Recovery: Server B ← Elastic IP (3.110.x.x) ← same IP!

Result: Your DNS, firewall rules, partner whitelists – nothing breaks ✅

IP Types Side-by-Side

Feature	● Private IP	● Public IP	● Elastic IP
Who Assigns	AWS automatically	AWS automatically	You request it
Survives Stop/Start	✓ Yes	✗ No — changes	✓ Yes
Internet Accessible	✗ No	✓ Yes	✓ Yes
Cost	Free	Free while running	Free if attached and running
Use Case	Internal VPC traffic	Temporary dev/test access	Production servers, DNS
Can Be Static	N/A (already stable)	✗ No	✓ Yes

Hands-On: Allocate and Use an Elastic IP

Step-by-Step

1. Allocate an Elastic IP

- EC2 Dashboard → Network & Security → **Elastic IPs**
- Click "**Allocate Elastic IP address**"
- Network border group: Leave default (your current region)
- Click "**Allocate**"
- AWS assigns you a static IP like **3.110.x.x** — this is now yours

2. Associate the EIP with Your Instance

- Select the newly allocated EIP
- **Actions** → **Associate Elastic IP address**

Field	Value
Resource type	Instance
Instance	Select your EC2 instance
Private IP	Select the primary private IP

- Click "**Associate**"
- Go to EC2 → Instances → your instance now shows the EIP as its public IPv4 ✓

3. Verify the EIP Sticks After Restart

```
# Note the current EIP
# Example: 3.110.45.67

# Stop the instance
# Wait 30 seconds
# Start the instance
```

```
# Check the public IP again in console
# It should still show: 3.110.45.67 ✅
# (contrast this with a regular public IP which would change)
```

4. Release an Elastic IP When Done

⚠️ Important – releasing unused EIPs saves money

```
EC2 → Elastic IPs → Select EIP
→ Actions → Disassociate (detach from instance first)
→ Actions → Release Elastic IP address
→ Confirm
```

The IP returns to AWS pool. Billing stops immediately.

⚖️ Auto Scaling Groups

💡 What is an Auto Scaling Group?

An **Auto Scaling Group (ASG)** is an AWS service that automatically manages the number of EC2 instances running your application. It watches your infrastructure, responds to changes in demand, and keeps your application available — all without any manual intervention.

Think of it like a smart staffing agency for your servers. When your website gets a traffic spike, the agency automatically hires more servers. When traffic drops at night, it lets some go. You define the rules, it handles the execution.

Traffic Pattern vs Instance Count:

9 AM – Office opens, traffic spikes
ASG detects CPU > 70%
Launches 2 new instances ↑

2 PM – Steady traffic
ASG maintains current count
No change needed

11 PM – Traffic drops overnight
ASG detects CPU < 30%
Terminates extra instances ↓

Result: You always have exactly what you need ✅
No over-provisioning, no under-provisioning

🔑 Core Concepts

Term	What It Means
Launch Template	The blueprint ASG uses to create each new instance (AMI, type, SG, startup script)
Desired Capacity	The number of instances ASG actively tries to maintain right now
Minimum Capacity	Hard floor — ASG will never go below this count, even at zero load
Maximum Capacity	Hard ceiling — ASG will never exceed this, even at peak load
Scaling Policy	The rules that decide when to add or remove instances
Health Check	ASG continuously checks instances — unhealthy ones get replaced automatically

Capacity Boundaries Visualized:

Maximum: 8  ← ASG never exceeds this
↓ scaling happens here
Desired: 4  ← Current target
↓ scaling happens here
Minimum: 2  ← ASG never drops below this

Scaling Policy Types

Policy	How It Decides to Scale	Best For
Target Tracking	Keeps a specific metric at your target (e.g., CPU stays at 50%)	Most common — simple and effective
Step Scaling	Scales by defined amounts at different alarm thresholds	Fine-grained control over scale steps
Scheduled	Scales at specific times you define (e.g., 9 AM add 3, 10 PM remove 3)	Predictable traffic patterns
Predictive	Uses ML to forecast traffic and pre-scales ahead of time	Large applications with historical data

Target Tracking Example:

Target: CPU = 50%

CPU hits 75% → Too high → ASG adds instances → CPU drops back toward 50%
CPU drops 20% → Too low → ASG removes instances → CPU rises back toward 50%
CPU stays 50% → Perfect → ASG does nothing

Why Use an ASG?

Without ASG:

Traffic spike → site crashes
Low traffic → paying for idle
Instance dies → site is down
You manually scale at 3am
Fixed monthly cost

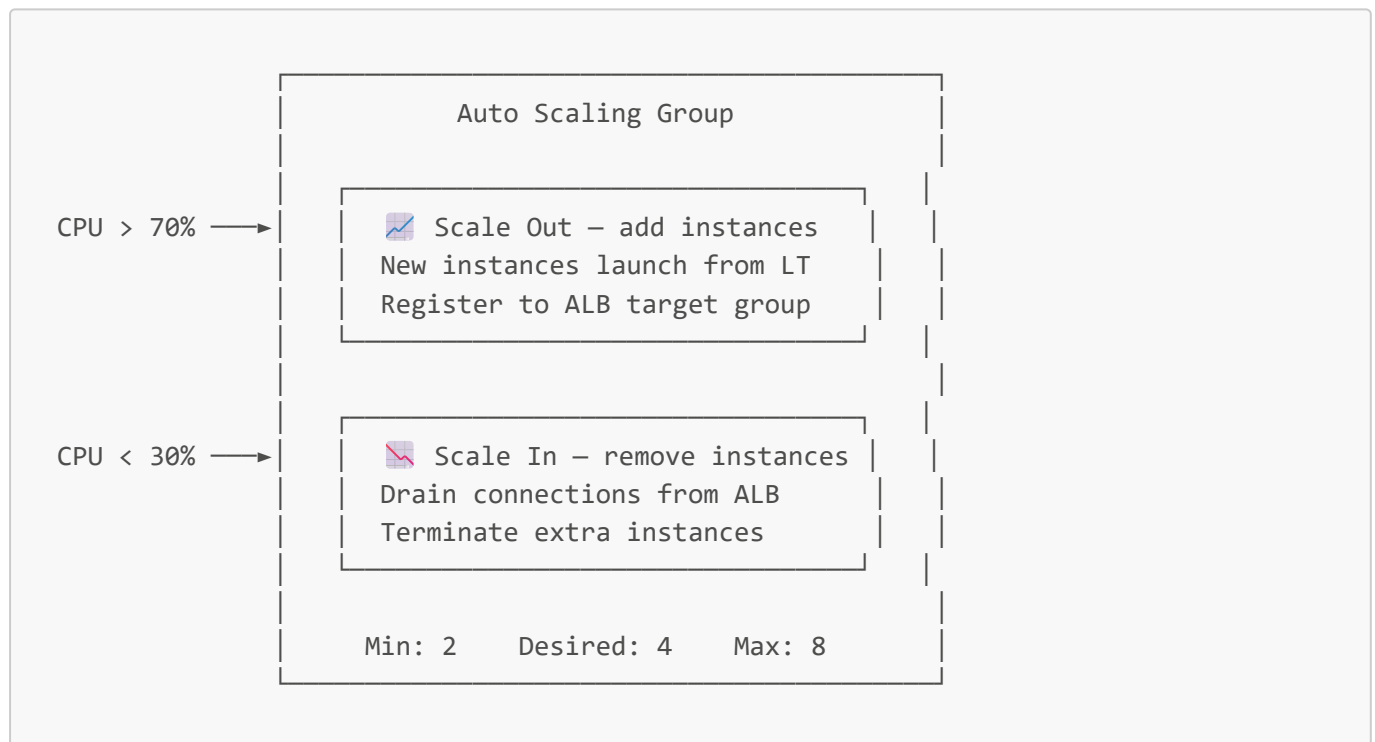
With ASG:

Traffic spike → new instances launch ✓
Low traffic → idle instances removed ✓
Instance dies → replaced in minutes ✓
ASG scales while you sleep ✓
Pay only for what you use ✓

Four Core Benefits:

-  **High Availability** — Failed instances are detected and replaced automatically without any downtime
-  **Cost Efficiency** — You scale in during quiet periods and scale out only when demand requires it
-  **Zero Manual Work** — Load-based decisions happen automatically based on your defined rules
-  **ALB Integration** — New instances register themselves to target groups and start receiving traffic immediately

⚙️ ASG Scaling Flow Diagram



🚀 Hands-On: Create a Launch Template

A Launch Template is the instruction set ASG follows every time it needs to create a new instance. Get this right and every auto-launched instance is identical and production-ready from second one.

What the Launch Template Defines

Launch Template: web-server-lt

AMI → Which OS + software image
Instance Type → How powerful each server is
Key Pair → SSH access credentials
Security Group → Firewall rules
Storage → Disk size and type
User Data → Startup script that runs on first boot

Step-by-Step

1. Open Launch Templates

- EC2 Dashboard → Left sidebar → **Launch Templates**
- Click "**Create launch template**"

2. Template Details

Field	Value
Launch template name	web-server-lt
Version description	v1 — Apache web server
Auto Scaling guidance	☒ Check this box

3. AMI and Instance

Field	Value
AMI	Amazon Linux 2023 (Free Tier Eligible)
Instance type	t3.small
Key pair	Select your existing key pair

4. Network Settings

Field	Value
Subnet	Do NOT specify — ASG will choose
Security groups	Select web-sg

Leaving subnet blank lets the ASG distribute instances across multiple AZs — critical for high availability.

5. Storage

Field	Value
Volume type	gp3

Field	Value
Size	8 GiB

6. User Data Script (Advanced Details → User Data)

```
#!/bin/bash

# Update system packages
apt update -y

# Install Apache web server
apt install -y apache2

# Start and enable Apache
systemctl start apache2
systemctl enable apache2

# Fetch instance ID from metadata service (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)

# Create a page that shows which instance is serving the request
echo "<h1>Response from Instance: $INSTANCE_ID</h1>" > /var/www/html/index.html
```

What this script does: Every time ASG launches a new instance, this script runs automatically. It installs Apache, starts it, and creates a webpage showing the instance ID — so you can visually confirm load balancing is working.

- Click **"Create launch template"** 

 Hands-On: Create the Auto Scaling Group

Step-by-Step

Step 1 — Name and Launch Template

- EC2 → **Auto Scaling Groups** → **Create Auto Scaling Group**

Field	Value
Auto Scaling group name	web-asg
Launch template	web-server-lt
Version	Latest (always uses newest template version)

Step 2 — Network Configuration

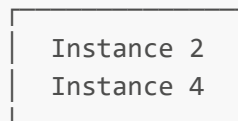
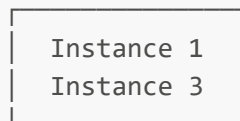
Field	Value
VPC	Your production VPC
Availability Zones / Subnets	<code>app-private-subnet-1a</code> AND <code>app-private-subnet-1b</code>

Always select **at least 2 subnets in different AZs**. If one AZ goes down, your instances in the other AZ keep serving traffic.

ASG Multi-AZ Layout:

AZ: `ap-south-1a`

AZ: `ap-south-1b`



ALB routes to both

If 1a goes down → 1b still serves all traffic ✓

Step 3 — Load Balancer Integration

Field	Value
Load balancing	Attach to an existing load balancer
Target group	Select <code>web-tg</code>
Health checks	✓ Enable Elastic Load Balancing health checks
Health check grace period	300 seconds

Why 300 seconds grace period? When a new instance launches, it needs time to finish the user data script, start Apache, and become ready. Without this grace period, ASG might mark it unhealthy before it's done booting and terminate it immediately.

Step 4 — Capacity and Scaling

Setting	Value
Desired capacity	2
Minimum capacity	2
Maximum capacity	6
Scaling policy type	Target tracking
Metric	Average CPU Utilization

Setting	Value
Target value	50%

What Target Tracking at 50% CPU means:

2 instances running, CPU hits 75% average:

- CloudWatch alarm triggers
- ASG launches 1-2 more instances
- Load spreads across more instances
- CPU average drops back toward 50%

Traffic decreases, CPU drops to 20%:

- ASG identifies excess instances
- Drains connections via ALB
- Terminates extra instances
- CPU rises back toward 50%

Step 5 — Notifications (Optional but Recommended)

- Add an SNS topic to receive email alerts when ASG launches or terminates instances
- Useful for auditing and keeping track of scaling events in production

Step 6 — Review and Create

- Review all settings
- Click **"Create Auto Scaling Group"** 

 Verify the ASG is Working

Check Scaling Activity:

EC2 → Auto Scaling Groups → Select web-asg → Activity tab

You should see entries like:

-  Launching instance i-0abc123 – Successful
-  Launching instance i-0def456 – Successful

Check Instances Were Created:

EC2 → Instances

Filter by ASG name tag → web-asg

You should see 2 instances in Running state
Both should be in different AZs (1a and 1b)

Check Target Group Registration:

EC2 → Target Groups → web-tg → Targets tab

Both instances should appear as:

Status: Healthy 

Port: 80

Test Auto Scaling — Simulate a CPU Spike

This test confirms your scaling policy actually works by artificially driving up CPU and watching ASG respond.

SSH into one of the ASG instances:

```
ssh -i my-keypair.pem ec2-user@INSTANCE_PUBLIC_IP
```

Install the stress testing tool:

```
sudo yum install -y stress
```

Run CPU stress test:

```
# Stress 4 CPU cores for 5 minutes  
stress --cpu 4 --timeout 300
```

Watch what happens in the console:

1. CloudWatch picks up rising CPU metric
↓
2. CPU average crosses 50% threshold
↓
3. CloudWatch alarm state changes to ALARM
↓
4. ASG receives scale-out signal
↓
5. New instance launches from web-server-1t
↓
6. New instance registers to web-tg
↓

7. ALB starts routing traffic to it
- ↓
8. CPU load spreads → metric drops back toward 50%

Monitor in real time:

EC2 → Auto Scaling Groups → web-asg

→ Activity tab	← Watch new launch events appear
→ Monitoring tab	← Watch CPU metric graph
→ Instance management	← Watch new instances appear

After stress test ends:

CPU drops → alarm clears → scale-in eventually occurs
(scale-in has a cooldown period – usually 5-15 min after CPU drops)

ASG Quick Reference

Key Numbers to Set:

Min = the floor you never drop below (for HA, set ≥ 2)

Max = the ceiling you never exceed (controls cost)

Desired = where you start right now

Timing to Know:

Health check grace period: 300 sec (let instance finish booting)

Scale-out cooldown: ~60-300 sec (wait before adding more)

Scale-in cooldown: ~300 sec (wait before removing – prevents flapping)

Always Do:

- Spread across 2+ AZs (never single AZ for production)
- Attach to a Target Group (ALB handles traffic distribution)
- Enable ELB health checks (not just EC2 health checks)
- Set meaningful Min ≥ 2 (single instance = single point of failure)

Avoid:

- Min = 0 (application goes completely down during low traffic)
- Single AZ deployment (one AZ outage = full outage)
- No scaling policy (defeats the purpose of ASG)
- Grace period too short (instances get terminated before they're ready)

Quick Reference Summary

Service Overview

Service	One-Line Purpose
EC2	Rent virtual servers in the cloud, pay per second
Security Groups	Virtual firewall — control who can reach your instance
AMI	Saved server template — launch identical instances anytime
EBS	Persistent virtual hard drive attached to EC2
Snapshots	Point-in-time EBS backup stored in S3
ENI	Virtual network card that connects instance to VPC
Elastic IP	Your own static public IP that never changes
Autoscaling Groups	Increases resources based on requirements automatically

💡 Key Rules to Remember

-  **EC2**
 - └ Always launched from an AMI
 - └ t2.micro is free tier – use it for learning
-  **Security Groups**
 - └ Stateful – allow inbound, response is auto-allowed
 - └ Allow-only – no DENY rules
 - └ Changes apply instantly, no restart needed
-  **AMI**
 - └ Region-specific – copy to deploy in other regions
 - └ Create BEFORE making major changes (safety net)
-  **EBS**
 - └ Locked to one AZ – must match EC2's AZ
 - └ Data persists after stop/start (not termination by default)
 - └ gp3 = best default choice for most workloads
 - └ Can only grow in size, never shrink
-  **Snapshots**
 - └ Incremental after first full backup
 - └ Stored in S3 (managed by AWS, not visible to you)
 - └ Can copy across regions for disaster recovery
-  **IP Addresses**
 - └ Private IP – stable, internal only, always present
 - └ Public IP – free but changes on every stop/start
 - └ Elastic IP – static, yours to keep, release when unused
 - └ Unused EIP = money wasted (\$0.005/hr)

AWS VPC — Networking Deep Dive & Production Architecture

Module: AWS Networking | **Level:** Beginner to Production **Goal:** Build a solid understanding of VPC concepts, then deploy a complete three-tier production architecture on AWS.

Table of Contents

1. [What is a VPC?](#)
2. [Public Subnet](#)
3. [Private Subnet](#)
4. [Internet Gateway](#)
5. [NAT Gateway](#)
6. [Bastion Host](#)
7. [Route Tables](#)
8. [Security Groups](#)
9. [Production Task — Three-Tier Architecture](#)

1. What is a VPC?

◆ Definition

A **Virtual Private Cloud (VPC)** is a logically isolated network that you create inside the AWS Cloud. It gives you complete control over your own networking environment — IP ranges, subnets, routing rules, and security settings — all within AWS infrastructure.

A simple way to think about it: imagine renting an entire floor of a shared office building. The building is AWS, but your floor is completely separated from everyone else. You decide where the walls go, which rooms connect to each other, and who gets a key to the front door.

When you create an AWS account, AWS automatically creates a **Default VPC** in every region. However, for real production workloads, you always create a **Custom VPC** where you define everything yourself.

◆ How AWS Handles It

- You assign your VPC a **CIDR block** — a range of private IP addresses your resources will use
- Everything inside stays **completely private by default** — no internet access unless you set it up
- A VPC is **region-scoped** — it spans across all Availability Zones in that region automatically
- You can run multiple VPCs per region (default limit is 5, which can be raised)
- VPCs can be connected to each other using **VPC Peering** or **AWS Transit Gateway**

◆ Why VPC Matters

Without a Custom VPC	With a Custom VPC
Shared address space with defaults	Full control over IP address design

Without a Custom VPC	With a Custom VPC
Default routing you can't fully customize	Custom route tables per subnet
Limited security segmentation	Multiple security layers (SGs, NACLs)
Not suitable for enterprise workloads	Production-ready, audit-friendly setup

◆ Understanding CIDR Blocks

CIDR (Classless Inter-Domain Routing) defines how large your IP address pool is.

VPC CIDR: 10.0.0.0/16

Breaking it down:

/16 = first 16 bits are fixed (the "10.0" part)

Remaining 16 bits are yours to assign

Total available IPs = 65,536

You slice this into smaller subnets:

10.0.1.0/24 → 256 IPs → Web Subnet AZ-1a

10.0.2.0/24 → 256 IPs → Web Subnet AZ-1b

10.0.3.0/24 → 256 IPs → App Subnet AZ-1a

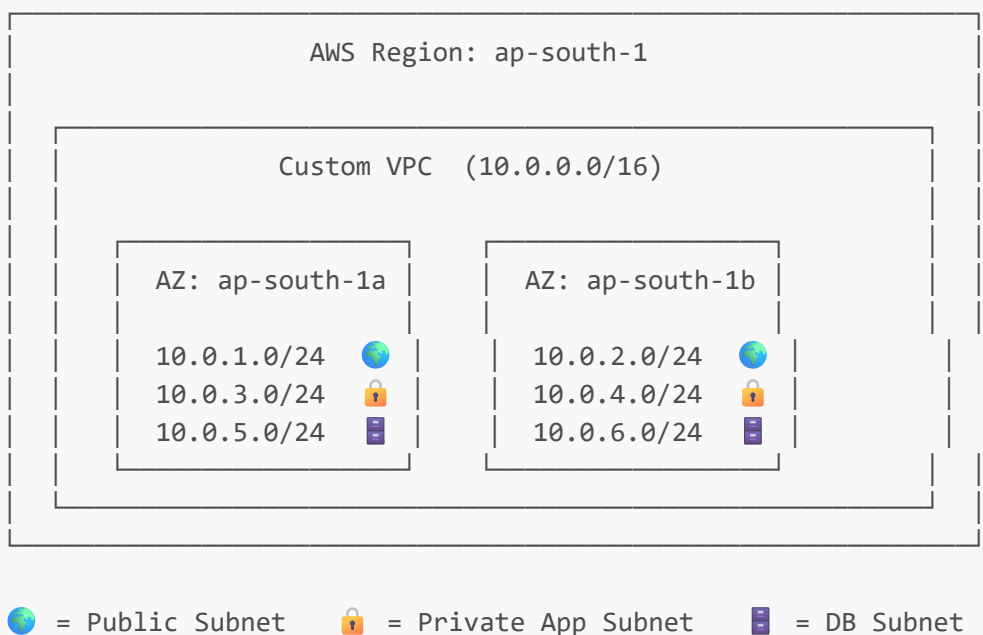
10.0.4.0/24 → 256 IPs → App Subnet AZ-1b

10.0.5.0/24 → 256 IPs → DB Subnet AZ-1a

10.0.6.0/24 → 256 IPs → DB Subnet AZ-1b

Starting with a /16 gives you plenty of room to grow without having to redesign your network later.

◆ VPC Layout Diagram



2. Public Subnet

● Definition

A **Public Subnet** is a subnet that has a direct route to the Internet Gateway in its route table. Resources placed here can be assigned public IPv4 addresses and can communicate both inward and outward with the internet.

Think of a public subnet like the lobby of an office building — it's the part that faces the street and is accessible to anyone coming in from outside.

● Defining Characteristics

- Route table contains: `0.0.0.0/0 → Internet Gateway`
- Instances can receive **public IPv4 addresses** (auto-assign setting must be enabled)
- Traffic can flow **in both directions** — from the internet to the resource and back
- Used only for resources that genuinely **need to be reachable from outside**

● What Belongs in a Public Subnet?

Resource	Reason It Needs Public Access
Application Load Balancer (ALB)	Receives HTTP/HTTPS traffic from internet users
NAT Gateway	Must reach the internet to forward private instance traffic
Bastion Host	Needs to be SSH-accessible from developer machines
Web-facing NGINX	Serves frontend content directly to browsers

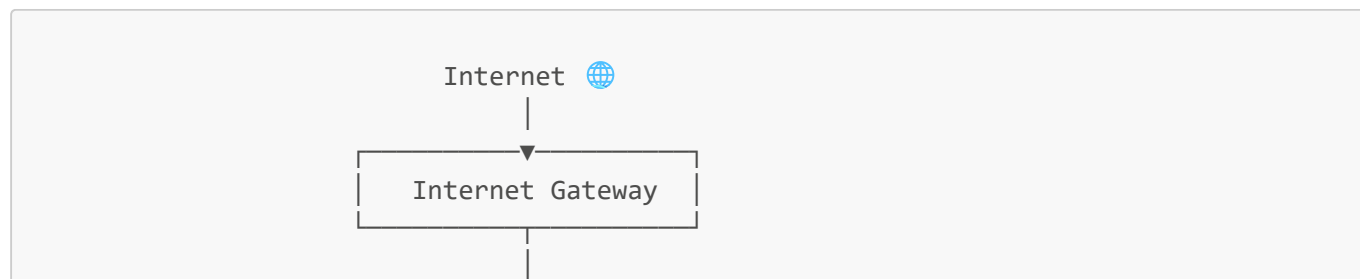
● Important AWS Detail — Reserved IPs

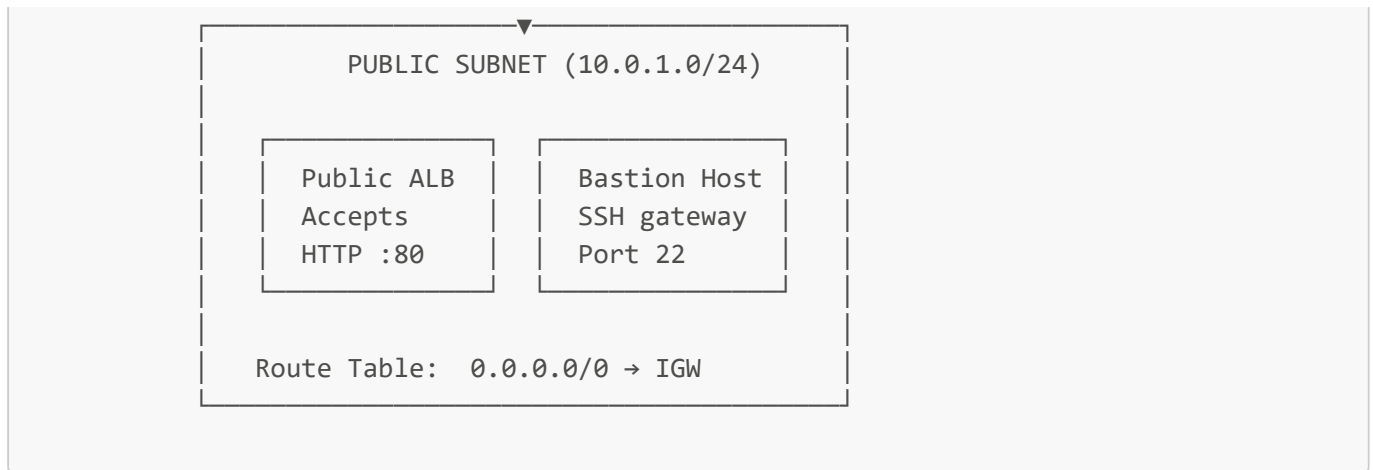
AWS reserves **5 IP addresses** in every subnet. For a `/24` subnet (256 total):

```
10.0.1.0   → Network address (reserved)
10.0.1.1   → VPC Router (AWS internal)
10.0.1.2   → AWS DNS server
10.0.1.3   → Reserved for future AWS use
10.0.1.255 → Broadcast address (reserved)
```

Usable IPs: $256 - 5 = 251$

● Public Subnet Traffic Flow





3. Private Subnet

● Definition

A **Private Subnet** is a subnet with no direct route to the Internet Gateway. Instances here only receive private IP addresses and are completely unreachable from the public internet. Any outbound internet traffic must flow through a **NAT Gateway** placed in a public subnet.

Using the office building analogy — private subnets are the inner offices and server rooms. Nobody from outside can walk directly in. Visitors must go through the lobby (public subnet) first.

● Why Private Subnets Are Critical

The security principle is simple: **if attackers cannot reach your servers, they cannot attack them**. Keeping your application logic and database in private subnets eliminates an entire category of attack vectors.

Even if your web tier gets compromised, the attacker still cannot directly access your database because it has no public IP and no route to the internet from outside.

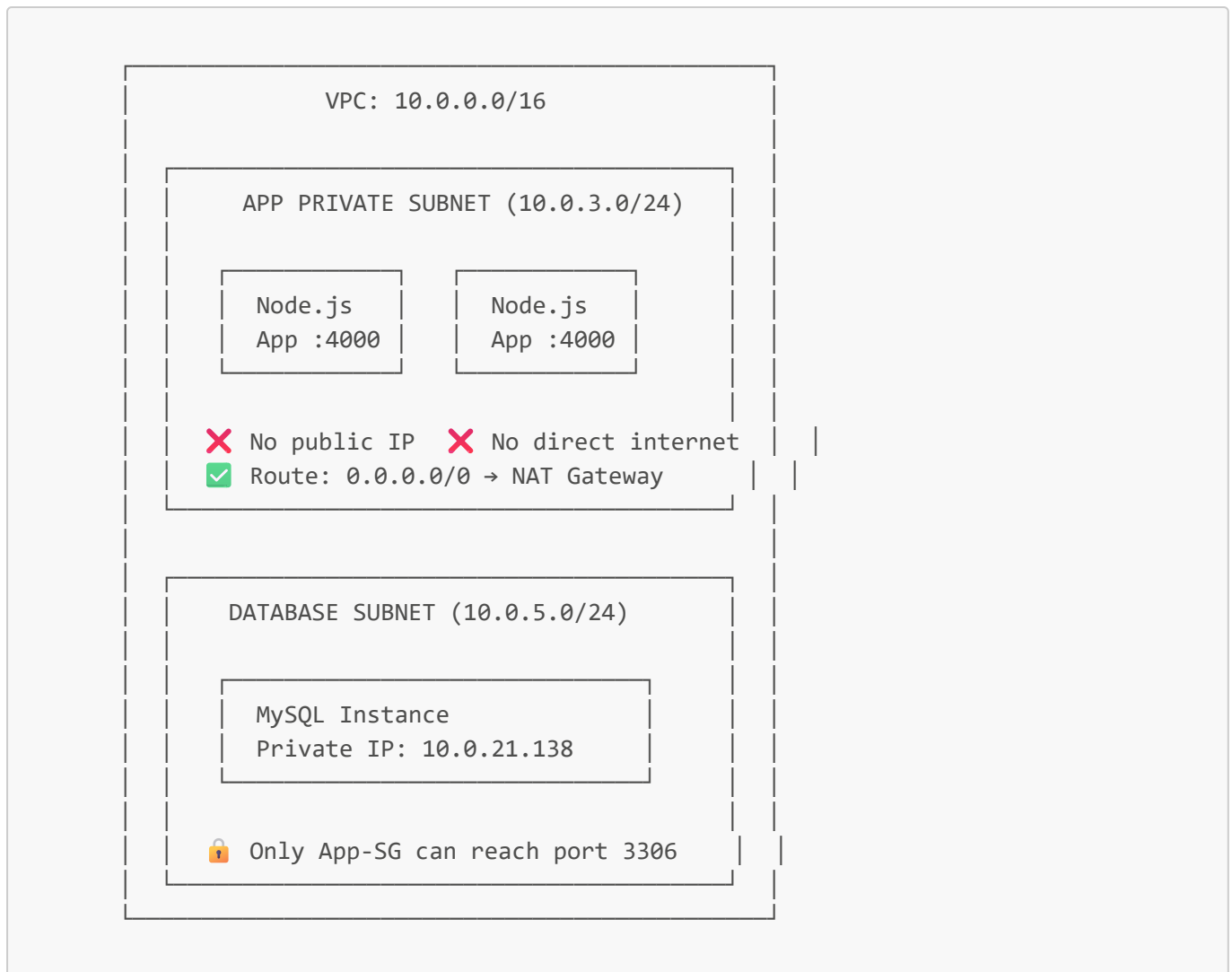
● Private Subnet Categories

Subnet Type	Hosts	Example CIDR
🔒 App Private Subnet	Node.js / backend API servers	10.0.3.0/24, 10.0.4.0/24
🗄️ Database Private Subnet	MySQL / RDS / database instances	10.0.5.0/24, 10.0.6.0/24

● Private Subnet Networking Rules

- No route to Internet Gateway — completely inbound-blocked from internet
- Outbound internet (for OS updates, package installs) goes through **NAT Gateway**
- Inbound connections only accepted from within the VPC — other subnets, load balancers, bastion host
- Database subnets are typically even more restricted — only the App tier can connect

● Private Subnet Diagram



4. Internet Gateway

🌐 Definition

An **Internet Gateway (IGW)** is a managed AWS component that you attach to your VPC to enable internet connectivity. It performs two specific functions:

1. Acts as a **routing target** in route tables for internet-bound traffic
2. Handles **Network Address Translation (NAT)** — translating the public IP of an instance to its private IP and back

Without an IGW attached to your VPC, no traffic can flow between your VPC and the public internet — even if instances have public IPs assigned.

🌐 Key Facts

- **One per VPC** — you can only attach a single IGW to a VPC at a time
- **Fully managed** — AWS handles all availability, capacity, and scaling automatically
- **No bandwidth cap** — it scales to handle any traffic volume without configuration
- **No cost to create** — you only pay for data transfer through it
- Must be explicitly **attached** to a VPC after creation — creating it alone does nothing

🌐 How Public IP Translation Works

Scenario: User accesses your website

User IP: 203.0.113.55



Hits IGW → IGW translates destination to instance private IP



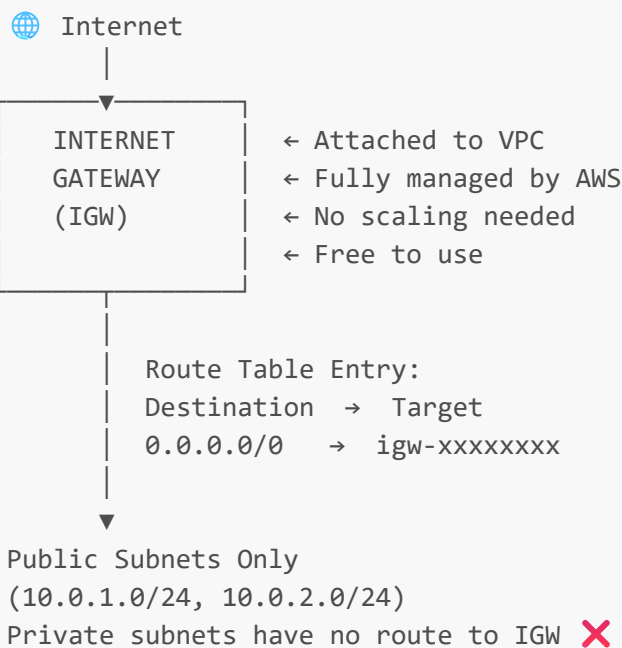
Instance Private IP: 10.0.1.25

(Instance has a Public IP mapped to it by AWS)

Return path: same translation in reverse

10.0.1.25 → IGW → 203.0.113.55

🌐 IGW Architecture Diagram



🌐 IGW vs NAT Gateway Comparison

Feature	Internet Gateway	NAT Gateway
Traffic Direction	Inbound + Outbound	Outbound only
Used By	Public subnets	Private subnets
Cost	Free	Paid (hourly + data GB)
Requires Public IP on instance?	Yes	No (NAT has its own Elastic IP)
Managed by AWS?	Yes	Yes

5. NAT Gateway

Definition

A **NAT Gateway (Network Address Translation Gateway)** is a managed AWS service that allows instances in private subnets to initiate outbound connections to the internet while ensuring the internet cannot initiate connections back to those instances.

It works by replacing the private IP of your instance with its own public Elastic IP when sending traffic outward. From the internet's perspective, all traffic appears to come from the NAT Gateway — your private instances remain completely hidden.

Why Private Instances Still Need Internet Access

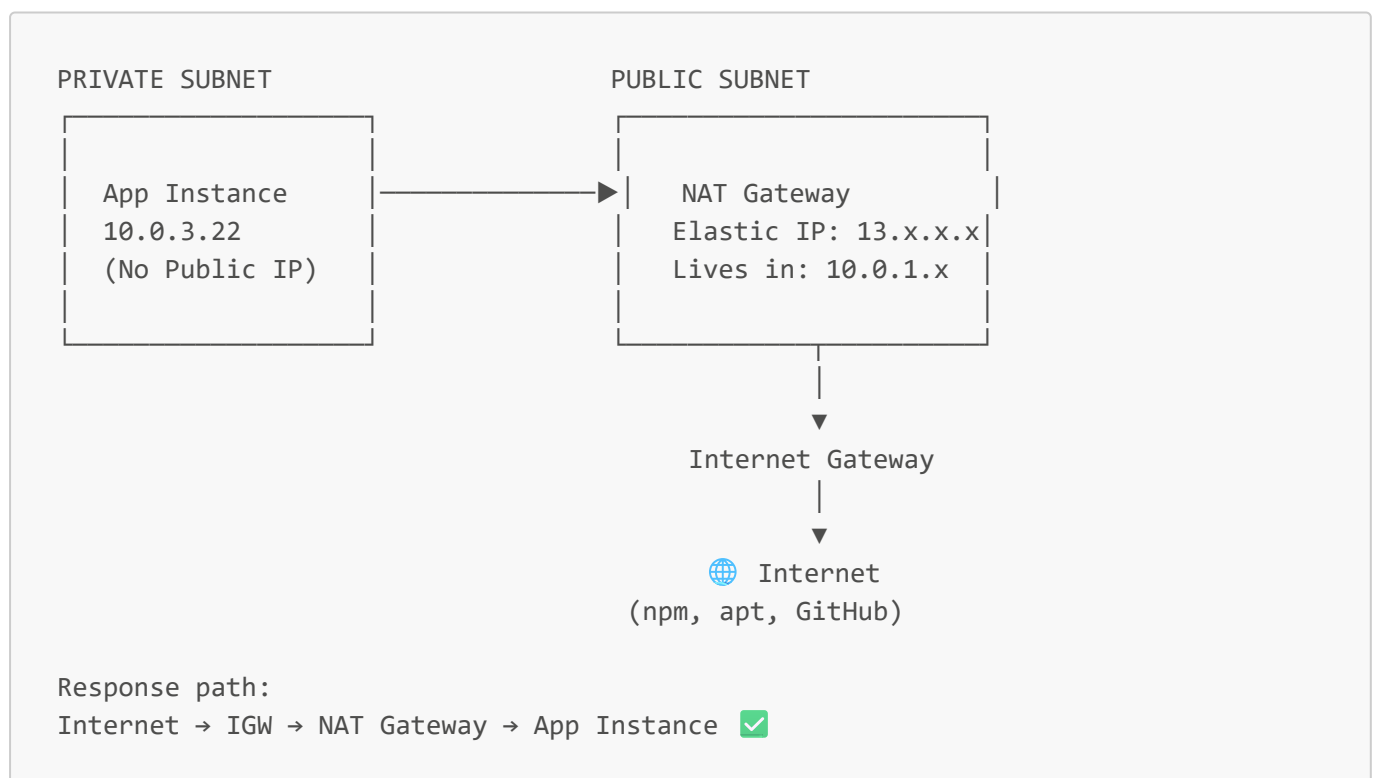
Even though your app servers and databases are private, they still need to reach the internet for operational reasons:

Common outbound needs from private instances:

- | | |
|-------------------|---|
| — sudo apt update | → Download OS security patches |
| — npm install | → Pull Node.js packages from npmjs.com |
| — git clone | → Fetch application code from GitHub |
| — pip install | → Python package downloads |
| — API calls | → Third-party services (Stripe, SendGrid, etc.) |

NAT Gateway handles all of this while keeping the instances invisible to inbound internet traffic.

NAT Gateway Traffic Flow



Important Deployment Notes

- NAT Gateway **must be placed in a public subnet** — it needs internet access itself
- Each NAT Gateway lives in **one Availability Zone** — for multi-AZ high availability, create one NAT Gateway per AZ
- Assign it an **Elastic IP** (static public IP) during creation
- Private route tables must point **0.0.0.0/0** to the NAT Gateway

Multi-AZ NAT Setup (Recommended for Production):

AZ-1a: NAT-GW-1 (EIP: 13.0.0.1) ← used by private subnets in 1a

AZ-1b: NAT-GW-2 (EIP: 13.0.0.2) ← used by private subnets in 1b

If AZ-1a fails, private instances in 1b still have internet access ✓

Single NAT setup: if that AZ fails, ALL private internet access breaks ✗

☑ NAT Gateway vs NAT Instance

Feature	NAT Gateway ✓	NAT Instance ⚠
Management	Fully AWS managed	You manage the EC2
Availability	High availability within AZ	Single point of failure
Max Bandwidth	Up to 45 Gbps	Limited by EC2 type
Setup Effort	Minimal	Requires manual config
Cost	Higher	Cheaper (EC2 pricing)
Recommended For	All production workloads	Dev/lab environments only

6. Bastion Host

🛡 Definition

A **Bastion Host** (sometimes called a Jump Server or Jump Box) is a dedicated EC2 instance placed in a public subnet that acts as a secure, controlled entry point for SSH access to instances in private subnets.

Because your app and database servers live in private subnets with no public IP, you cannot SSH into them directly. The Bastion Host bridges this gap — you SSH into the Bastion first, then SSH onward to your private instance from there.

🛡 The Security Problem It Solves

Without Bastion Host:

Developer Laptop → ??? → Private App Server

No route exists. Cannot connect. ✗

With Bastion Host:

Developer Laptop → Bastion Host → Private App Server ✓
Controlled, logged, single entry point.

Security Best Practices

Practice	Why It Matters
Restrict SSH source IP to your own IP	Prevents the entire internet from attempting logins
Always use key pairs, never passwords	Keys are cryptographically stronger than passwords
Enable AWS CloudTrail logging	Creates an audit trail of who logged in and when
Stop the Bastion when not in use	Reduces attack window and saves EC2 cost
Consider AWS Systems Manager Session Manager	Zero-bastion alternative — no open port 22 needed

Connecting Through the Bastion

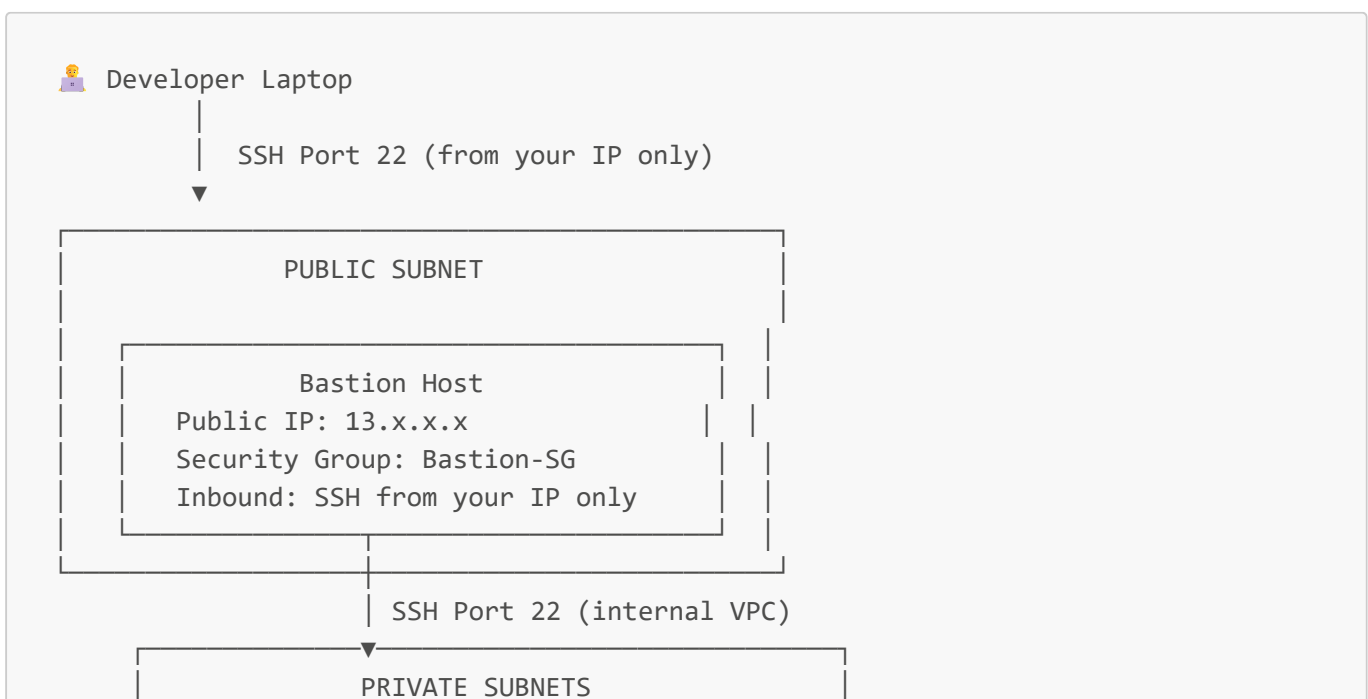
```
# Method 1 – Two separate SSH hops

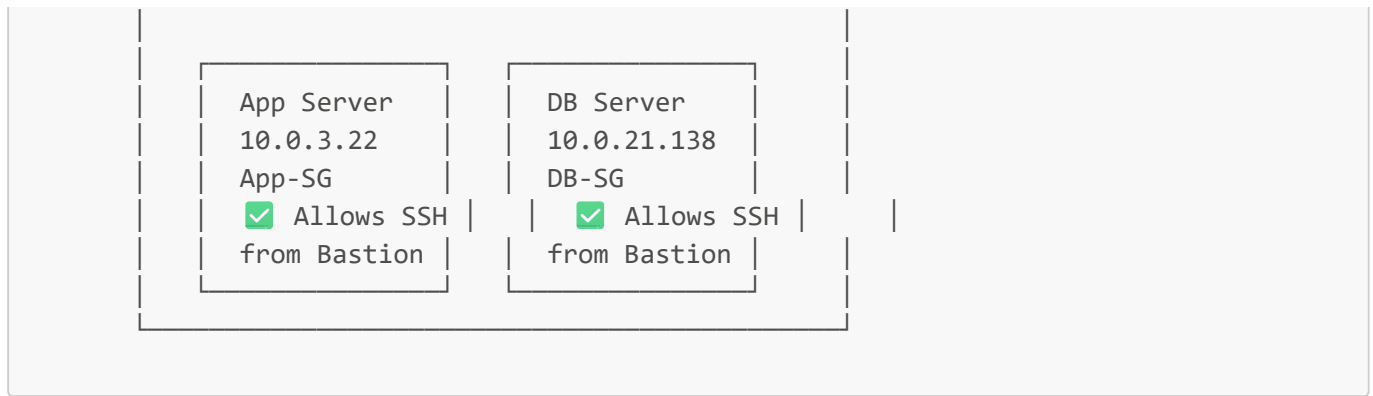
# Hop 1: Connect to Bastion (it has a public IP)
ssh -i mykey.pem ubuntu@<bastion-public-ip>

# Hop 2: From inside Bastion, connect to private instance
ssh -i mykey.pem ubuntu@10.0.3.22

# Method 2 – Single command using ProxyJump (-J flag)
ssh -i mykey.pem -J ubuntu@<bastion-public-ip> ubuntu@10.0.3.22
```

Bastion Host Access Diagram





7. Route Tables

Definition

A **Route Table** is a set of rules — called routes — that tells your VPC where to send network traffic based on its destination. Every subnet in your VPC must be associated with exactly one route table, which governs where traffic from that subnet gets forwarded.

Think of route tables like a GPS system for your network packets. Each packet has a destination IP, and the route table is the map that says "to reach that destination, go through this gateway."

How AWS Picks a Route

When traffic leaves a subnet, AWS checks the route table and picks the route with the **most specific match** (longest prefix):

Traffic going to 10.0.3.55 — which route wins?

Route 1: 10.0.0.0/16 → local (matches — 16 bits specific)

Route 2: 0.0.0.0/0 → IGW (matches — 0 bits specific)

Winner: Route 1 (10.0.0.0/16) — more specific = longer prefix

Result: Traffic stays inside the VPC ✓

Three Route Tables in This Architecture

Public Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	igw-xxxxxxx	← Everything else goes to internet

Associated with: [web-public-subnet-1a](#), [web-public-subnet-1b](#)

Private App Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	nat-xxxxxxx	← Outbound internet goes via NAT GW

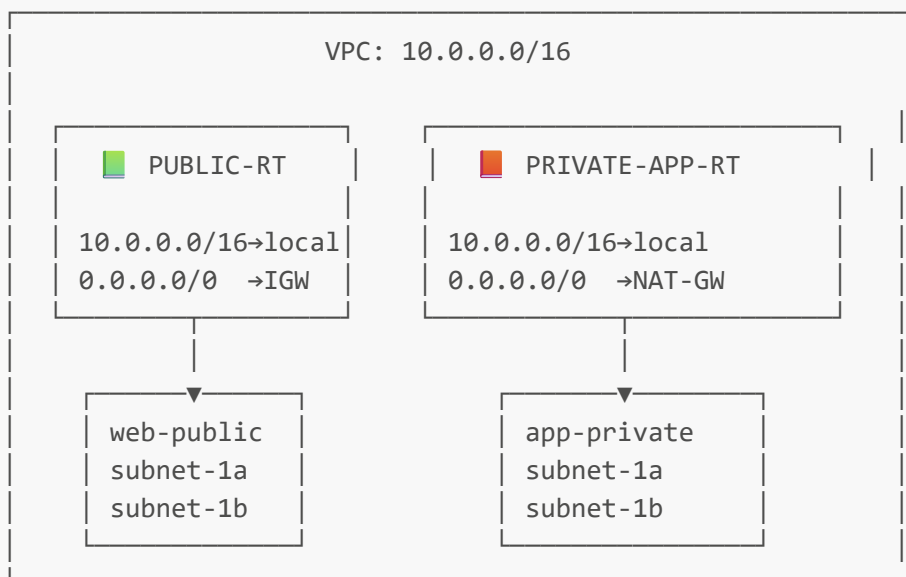
Associated with: `app-private-subnet-1a`, `app-private-subnet-1b`

Database Route Table

Destination	Target	
10.0.0.0/16	local	← Internal VPC traffic stays inside
0.0.0.0/0	nat-xxxxxxx	← Outbound internet goes via NAT GW

Associated with: `data-private-subnet-1a`, `data-private-subnet-1b`

Route Table Visual Map



The Local Route — Always Present

Every route table contains a `local` route for the VPC CIDR block. This route:

- **Cannot be deleted or modified**
- Allows all subnets within the VPC to communicate with each other freely
- Does not go through any gateway — traffic stays within AWS's internal fabric

8. Security Groups

🔒 Definition

A **Security Group** is a virtual firewall that controls what traffic is allowed to reach (inbound) and leave (outbound) your EC2 instances. It operates at the **instance level**, meaning each instance can have different security groups applied to it.

Unlike traditional firewalls that work at the network boundary, Security Groups attach directly to individual instances — giving you precise per-resource control.

🔒 Core Characteristics

- **Stateful** — if an inbound connection is allowed, the response traffic is automatically permitted without needing a separate outbound rule. This is different from NACLs which are stateless.
- **Allow-only rules** — you can only create ALLOW rules. There is no explicit DENY — anything not explicitly allowed is automatically blocked.
- **Reference other Security Groups as sources** — instead of specifying an IP range, you can say "allow traffic from any instance that has Security Group X." This is the most powerful feature for layered architectures.
- Applied at the **Elastic Network Interface (ENI)** level
- Rule changes apply **immediately** — no restart required

🔒 Stateful vs Stateless (Quick Clarification)

Stateful (Security Groups):

Inbound rule: ALLOW port 80 from 0.0.0.0/0

→ Response traffic (outbound) is automatically allowed ✅

→ No extra outbound rule needed

Stateless (NACLs – Network Access Control Lists):

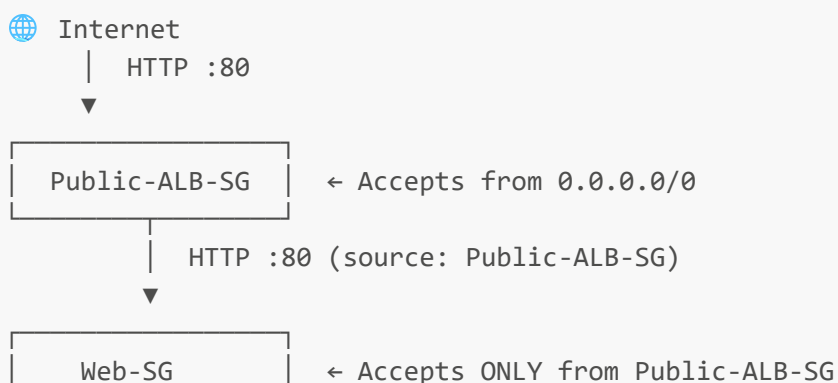
Inbound rule: ALLOW port 80

→ You must ALSO add an outbound rule to allow the response ⚠️

→ Both directions must be explicitly configured

🔒 Security Group Chaining

This is the architecture that makes AWS networking truly secure. Each layer only accepts traffic from the layer directly above it, using SG references instead of IP ranges.





This creates a **zero-trust chain** — each layer is completely isolated and can only be reached from its direct upstream component.

9. Production Task — Three-Tier Architecture

What We're Building

A fully production-grade, highly available three-tier web application deployed across two Availability Zones with auto-scaling at every tier.

Architecture Stack:

-  Web Tier → NGINX serving React frontend + reverse proxy
-  App Tier → Node.js REST API running on port 4000
-  DB Tier → MySQL database instance (isolated)

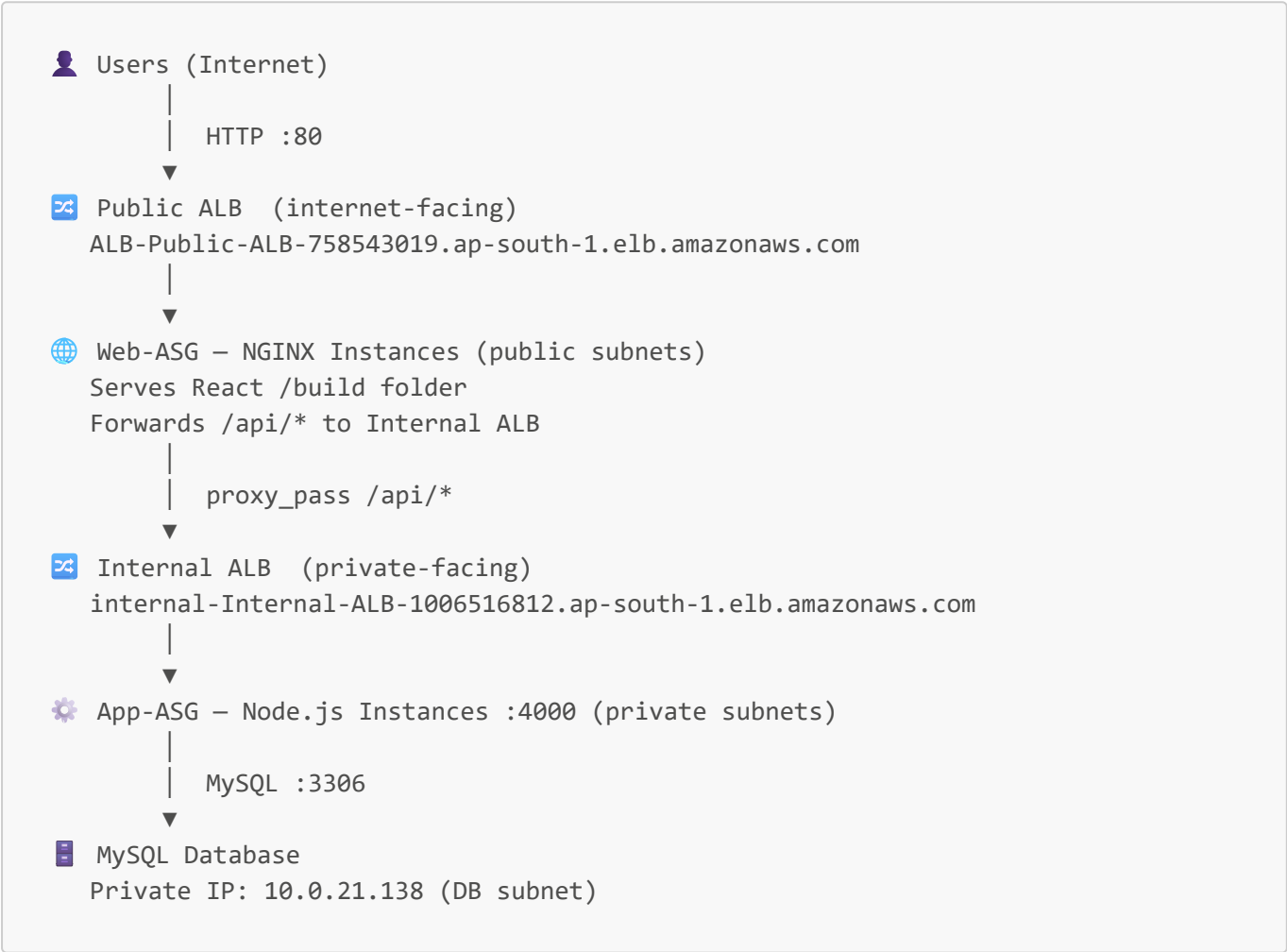
Infrastructure Details

Component	Value
VPC CIDR	10.0.0.0/16
Region	ap-south-1 (Mumbai)
Availability Zones	ap-south-1a and ap-south-1b
Public ALB DNS	ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com
Internal ALB DNS	internal-Internal-ALB-1006516812.ap-south-1.elb.amazonaws.com

Subnet Layout

Subnet Name	Category	AZ	CIDR
web-public-subnet-1a	 Public	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	 Public	ap-south-1b	10.0.2.0/24
app-private-subnet-1a	 Private App	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	 Private App	ap-south-1b	10.0.4.0/24
data-private-subnet-1a	 Private DB	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	 Private DB	ap-south-1b	10.0.6.0/24

End-to-End Request Flow



Step-by-Step Implementation

Step 1 — Create the VPC

AWS Console → VPC → Your VPCs → Create VPC

Setting	Value
Name Tag	Production-VPC

Setting	Value
IPv4 CIDR	10.0.0.0/16
Tenancy	Default

✓ Step 2 — Create and Attach Internet Gateway

VPC → Internet Gateways → Create Internet Gateway

Setting	Value
Name Tag	Production-IGW

After creation: **Actions** → **Attach to VPC** → **Select Production-VPC** → **Attach**

Without attaching the IGW, your public subnets cannot reach the internet even with the right route table entries.

✓ Step 3 — Create Public Subnets

VPC → Subnets → Create Subnet → Select Production-VPC

Name	AZ	CIDR
web-public-subnet-1a	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	ap-south-1b	10.0.2.0/24

After creating each: **Select subnet** → **Actions** → **Edit Subnet Settings** → **Enable Auto-assign Public IPv4** ✓

✓ Step 4 — Create App Private Subnets

Name	AZ	CIDR
app-private-subnet-1a	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	ap-south-1b	10.0.4.0/24

Do NOT enable auto-assign public IP here. These stay private.

✓ Step 5 — Create Database Private Subnets

Name	AZ	CIDR
data-private-subnet-1a	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	ap-south-1b	10.0.6.0/24

✓ Step 6 — Create NAT Gateway

VPC → NAT Gateways → Create NAT Gateway

Setting	Value
Name	Production-NAT
Subnet	web-public-subnet-1a ← must be a public subnet
Elastic IP	Click "Allocate Elastic IP"

 Wait for NAT Gateway status to show **Available** before moving to route tables — usually takes 1-2 minutes.

Why this matters: Private subnet instances (app servers, database) need outbound internet access for:

```
apt update      → OS security patches
npm install     → Node.js dependencies
git clone       → Fetching application code
```

✓ Step 7 — Configure Route Tables

VPC → Route Tables → Create Route Table

Public Route Table

Setting	Value
Name	Public-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-IGW

Subnet Associations: [web-public-subnet-1a](#), [web-public-subnet-1b](#)

Private App Route Table

Setting	Value
Name	Private-App-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: [app-private-subnet-1a](#), [app-private-subnet-1b](#)

 Database Route Table

Setting	Value
Name	Database-RT
VPC	Production-VPC

Routes to add:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: [data-private-subnet-1a](#), [data-private-subnet-1b](#)

 Step 8 — Configure Security Groups

EC2 → Security Groups → Create Security Group

 Bastion-SG

Rule	Protocol	Port	Source
Inbound — SSH	TCP	22	0.0.0.0/0

 Public-ALB-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	0.0.0.0/0

 Web-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	Public-ALB-SG
Inbound — SSH	TCP	22	Bastion-SG

 Internal-ALB-SG

Rule	Protocol	Port	Source
Inbound — HTTP	TCP	80	Web-SG

 App-SG

Rule	Protocol	Port	Source
Inbound — Custom TCP	TCP	4000	Internal-ALB-SG
Inbound — SSH	TCP	22	Bastion-SG

 DB-SG

Rule	Protocol	Port	Source
Inbound — MySQL	TCP	3306	App-SG
Inbound — SSH	TCP	22	Bastion-SG

Security Group Assignment Map

Resource	Uses Security Group
Bastion Host	→ Bastion-SG
Public ALB	→ Public-ALB-SG
Web Instances	→ Web-SG
Internal ALB	→ Internal-ALB-SG
App Instances	→ App-SG
MySQL Database	→ DB-SG

 Step 9 — Launch Bastion Host

EC2 → Launch Instance

Setting	Value
Name	Bastion-Host
AMI	Ubuntu Server 22.04 LTS
Instance Type	t2.micro
Subnet	web-public-subnet-1a
Auto-assign Public IP	Enabled 

Setting	Value
Security Group	Bastion-SG

✔ Step 10 — Create Internal ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Setting	Value
Name	Internal-ALB
Scheme	Internal
Subnets	app-private-subnet-1a, app-private-subnet-1b
Security Group	Internal-ALB-SG

✔ Step 11 — Create App Target Group

EC2 → Target Groups → Create Target Group

Setting	Value
Name	App-TG
Protocol	HTTP
Port	4000
Target Type	Instance
Health Check Protocol	HTTP
Health Check Path	/health

✔ Step 12 — Launch Database EC2 Instance

EC2 → Launch Instance

Setting	Value
Name	DB-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	data-private-subnet-1a
Auto-assign Public IP	Disabled ✖
Security Group	DB-SG

✔ Step 13 — Install and Configure MySQL

SSH into DB instance through Bastion, then:

```
# System update
sudo apt update && sudo apt upgrade -y

# Install MySQL Server
sudo apt install mysql-server -y

# Enter MySQL shell
sudo mysql
```

Inside MySQL:

```
-- Create the application database
CREATE DATABASE webappdb;

-- Create dedicated app user (not using root)
CREATE USER 'appuser'@'%' IDENTIFIED BY 'YourStrongPassword123!';

-- Grant access only to the app database
GRANT ALL PRIVILEGES ON webappdb.* TO 'appuser'@'%';

-- Apply privilege changes
FLUSH PRIVILEGES;

EXIT;
```

Allow remote connections from App tier:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Find and change:

```
# Before (only accepts local connections):
bind-address = 127.0.0.1

# After (accepts from all interfaces – App-SG restricts who can actually reach
it):
bind-address = 0.0.0.0
```

Restart MySQL:

```
sudo systemctl restart mysql
```

```
sudo systemctl enable mysql
```

✓ Step 14 — Launch App Tier Test Instance

EC2 → Launch Instance

Setting	Value
Name	App-Test-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	app-private-subnet-1a
Auto-assign Public IP	Disabled ✖
Security Group	App-SG

✓ Step 15 — Install Node.js and PM2

SSH in via Bastion, then:

```
# System update
sudo apt update && sudo apt upgrade -y

# Add Node.js 16 package source
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -

# Install Node.js and Git
sudo apt install -y nodejs git

# Install PM2 globally (keeps Node.js app alive after terminal closes)
sudo npm install -g pm2
```

What is PM2? PM2 is a process manager for Node.js. It keeps your app running in the background, restarts it if it crashes, and can be configured to auto-start on server reboot. Essential for production Node.js deployments.

✓ Step 16 — Clone Application Code

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

✓ Step 17 — Set Database Configuration


```
nano aws-three-tier-web-architecture-workshop/application-code/app-  
tier/DbConfig.js
```

```
module.exports = Object.freeze({  
  DB_HOST      : '10.0.21.138',          // Private IP of your MySQL instance  
  DB_USER      : 'appuser',  
  DB_PWD       : 'YourStrongPassword123!',  
  DB_DATABASE  : 'webappdb'  
});
```

✓ Step 18 — Start the App Tier

```
cd aws-three-tier-web-architecture-workshop/application-code/app-tier  
  
# Install all Node.js dependencies  
npm install  
  
# Launch app with PM2  
pm2 start index.js --name app-tier  
  
# Save process list so it survives reboots  
pm2 save
```

Verify the app is running correctly:

```
curl http://localhost:4000/health
```

Expected output:

```
"This is the health check"
```

✓ Step 19 — Register App Instance to Target Group

1. **EC2 → Target Groups → App-TG → Register Targets**
2. Select the App Test Instance
3. Click **Include as pending below → Register pending targets**
4. Wait for health status to show **✓ Healthy**

✓ Step 20 — Create Public ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Setting	Value
Name	Public-ALB
Scheme	Internet-facing
Subnets	web-public-subnet-1a, web-public-subnet-1b
Security Group	Public-ALB-SG
Listener	HTTP :80 → forward to Web-TG

 Step 21 — Create Web Target Group**EC2 → Target Groups → Create Target Group**

Setting	Value
Name	Web-TG
Protocol	HTTP
Port	80
Target Type	Instance
Health Check Path	/

 Step 22 — Launch Web Tier Test Instance**EC2 → Launch Instance**

Setting	Value
Name	Web-Test-Instance
AMI	Ubuntu Server 22.04 LTS
Subnet	web-public-subnet-1a
Auto-assign Public IP	Enabled 
Security Group	Web-SG

 Step 23 — Install NGINX and Node.js on Web Tier

SSH directly into web instance (it has a public IP):

```
# System update
sudo apt update && sudo apt upgrade -y
```

```
# Add Node.js 16 repository
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -

# Install Node.js, NGINX, and Git together
sudo apt install nodejs nginx git -y
```

Clone the repository:

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

✓ Step 24 — Build the React Frontend

```
cd aws-three-tier-web-architecture-workshop/application-code/web-tier

# Install React dependencies
npm install

# Build production-optimized bundle
npm run build
```

This generates a `/build` directory containing compiled, minified static files that NGINX will serve.

✓ Step 25 — Configure NGINX

```
sudo nano /etc/nginx/sites-available/default
```

Replace the entire file with:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # Health check endpoint for ALB target group checks
    location /health {
        default_type text/html;
        return 200 "<!DOCTYPE html><p>Web Tier Health Check</p>\n";
    }

    # Serve compiled React frontend
    location / {
        root /home/ubuntu/aws-three-tier-web-architecture-workshop/application-code/web-tier/build;
    }
}
```

```
    index index.html index.htm;
    try_files $uri /index.html;
}

# Reverse proxy: all /api/ requests forwarded to Internal ALB → App Tier
location /api/ {
    proxy_pass http://internal-Internal-ALB-1006516812.ap-south-
1.elb.amazonaws.com;
}
}
```

How NGINX Bridges Web and App Tiers

```
Browser → GET /api/transactions
      |
      ▼
  NGINX on Web Instance
      |
  matches location /api/
      |
      ▼
  proxy_pass to Internal ALB
      |
      ▼
  Internal ALB → App-TG
      |
      ▼
  Node.js :4000 handles request
      |
  Response flows back same path
      ▼
  Browser receives data ✓
```

✓ Step 26 — Test and Reload NGINX

```
# Validate NGINX config before applying
sudo nginx -t

# Restart to apply changes
sudo systemctl restart nginx

# Enable auto-start on reboot
sudo systemctl enable nginx
```

✓ Step 27 — Register Web Instance to Web Target Group

1. **EC2 → Target Groups → Web-TG → Register Targets**
2. Select Web Test Instance → Register
3. Wait for health status:  **Healthy**

Step 28 — End-to-End Testing

Test the frontend:

```
http://ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com
```

Test API via web tier:

```
curl http://localhost/api/health
```

Expected:

```
"This is the health check"
```

Full stack test: Use the DB Demo page in the React app to verify read/write to MySQL through the full three-tier chain.

Step 29 — Create AMIs for Auto Scaling

Capture the current state of your tested instances as reusable images.

EC2 → Instances → Select Instance → Actions → Image and Templates → Create Image

Instance	AMI Name	Note
Web-Test-Instance	ubuntu-web-tier-ami-v1	Enable reboot for clean filesystem
App-Test-Instance	ubuntu-app-tier-ami-v1	Enable reboot for clean filesystem

Enabling "Reboot instance" before snapshot ensures all data is written to disk properly, giving you a consistent AMI.

Step 30 — Create Launch Templates

EC2 → Launch Templates → Create Launch Template

Web Launch Template

Setting	Value
Name	Web-LT
AMI	ubuntu-web-tier-ami-v1
Security Group	Web-SG

App Launch Template

Setting	Value
Name	App-LT
AMI	ubuntu-app-tier-ami-v1
Security Group	App-SG

Step 31 — Create Auto Scaling Groups

EC2 → Auto Scaling Groups → Create Auto Scaling Group

Web-ASG

Setting	Value
Name	Web-ASG
Launch Template	Web-LT
Subnets	web-public-subnet-1a, web-public-subnet-1b
Target Group	Web-TG
Desired	2
Minimum	2
Maximum	4

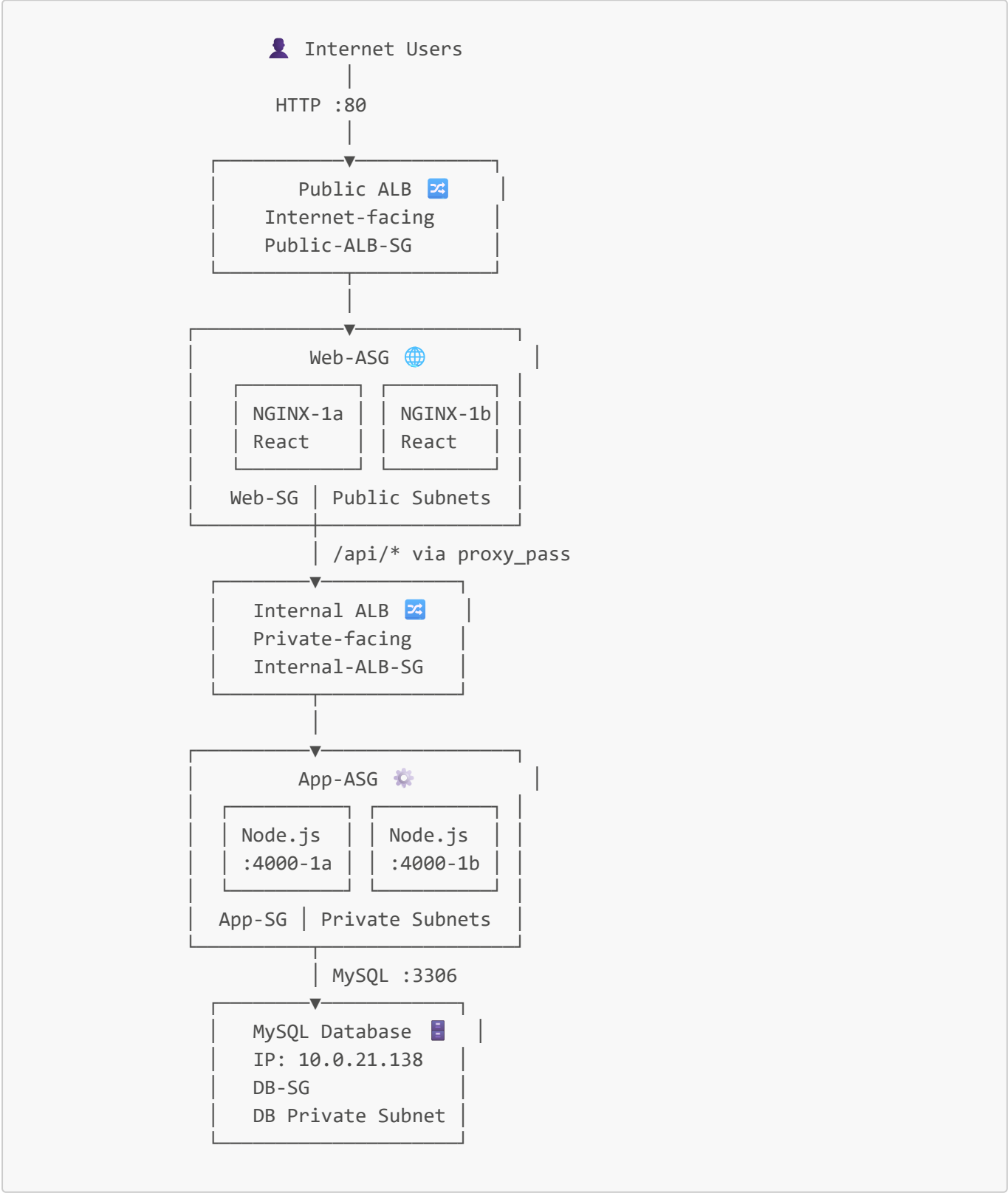
App-ASG

Setting	Value
Name	App-ASG
Launch Template	App-LT
Subnets	app-private-subnet-1a, app-private-subnet-1b
Target Group	App-TG
Desired	2
Minimum	2

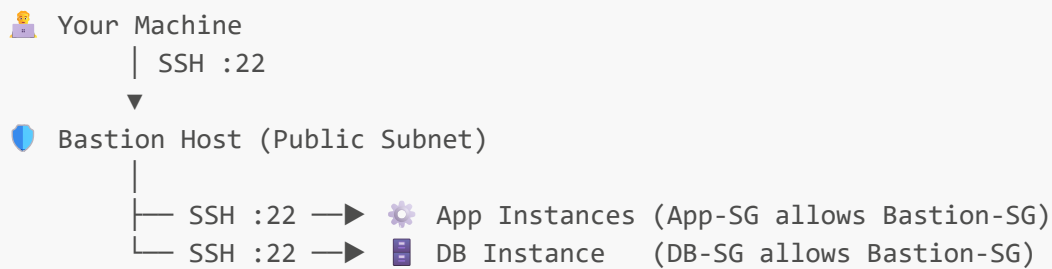
Setting	Value
Maximum	4

Final Architecture Summary

Complete Traffic Flow



SSH Access Path



Auto Scaling Summary

```

Web-ASG
Template : Web-LT
AMI      : web-tier-ami-v1
SG       : Web-SG
Subnets : public-1a, 1b
Capacity : 2 min / 4 max
Target   : Web-TG
  
```

```

App-ASG
Template : App-LT
AMI      : app-tier-ami-v1
SG       : App-SG
Subnets : private-1a, 1b
Capacity : 2 min / 4 max
Target   : App-TG
  
```

🏆 Production Benefits of This Architecture

✅ Benefit	🔧 How It's Achieved
High Availability	Every tier deployed across two Availability Zones
Fault Tolerance	Auto Scaling Groups replace failed instances automatically
Backend Security	App and DB servers have no public IP, not reachable from internet
Elastic Scalability	ASGs scale from 2 to 4 instances based on actual load
Controlled SSH Access	Single audited entry point via Bastion Host
Clean Separation of Concerns	NGINX handles frontend; Node.js handles API; MySQL handles data
Database Isolation	DB subnet only reachable from App-SG on port 3306
Dual Load Balancing	Two ALBs distribute load at web tier and app tier independently

 Benefit **How It's Achieved****Future-Proof IP Design**

/16 CIDR gives room for 65,536 IPs and hundreds of subnets

No Direct DB Internet Exposure

DB subnet has no IGW route — outbound-only via NAT

AWS Load Balancer – Complete Notes

 1. What is a Load Balancer?

Definition

A Load Balancer is a networking service that distributes incoming traffic across multiple targets (EC2 instances, containers, IP addresses, Lambda functions) in one or more Availability Zones to ensure:

- High Availability
- Fault Tolerance
- Scalability
- Reliability

 Real-World Analogy

Imagine a restaurant with multiple billing counters.

A manager stands at the entrance and sends customers to the counter with the ****shortest queue****.

That manager is exactly like a Load Balancer

Result:

- No single counter gets overloaded
- Customers are served faster
- If one counter fails, others continue working

 Key Benefits of Load Balancers

1. High Availability: Distributes traffic across multiple Availability Zones
2. Fault Tolerance: Stops sending traffic to unhealthy servers
3. Scalability: Handles increasing traffic automatically
4. Security: Backend servers are hidden behind the LB
5. Health Checks: Continuously monitors target health
6. SSL/TLS Termination: Offloads HTTPS encryption/decryption
7. Better Performance: Prevents server overload

 AWS Elastic Load Balancing (ELB) Family

AWS Elastic Load Balancing (ELB)

ALB Application LB Layer 7 HTTP/HTTPS	NLB Network LB Layer 4 TCP/UDP/TLS	GLB Gateway LB Layer 3 IP Traffic
--	---	--

Note: Classic Load Balancer (CLB) is legacy/deprecated. AWS recommends ALB or NLB for new projects.

2. Application Load Balancer (ALB) — Layer 7

Definition

An **Application Load Balancer (ALB)** operates at **OSI Layer 7 (Application Layer)**.

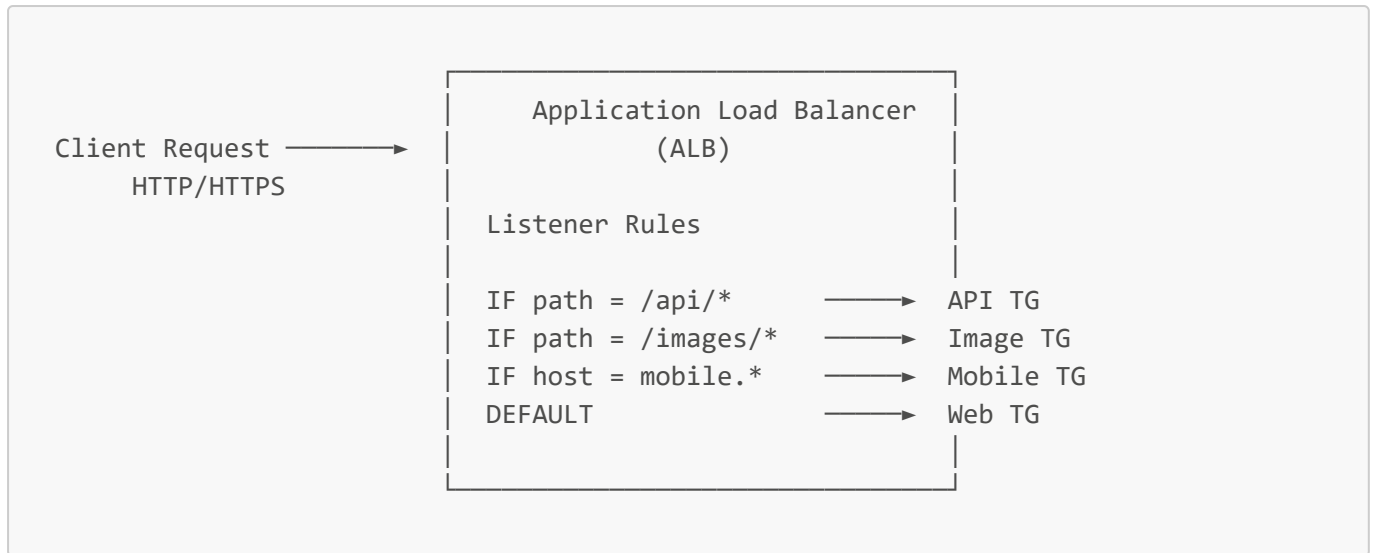
It makes routing decisions based on the **content of HTTP/HTTPS requests** such as:

- URL Path
- Hostname
- HTTP Headers
- Query Strings
- HTTP Methods

ALB Key Characteristics

Feature	Description
Protocol Support	HTTP, HTTPS, gRPC, WebSocket
OSI Layer	Layer 7 (Application Layer)
Routing Type	Content-Based Routing
Targets Supported	EC2, ECS, Lambda, IP addresses
SSL Termination	Supported
Sticky Sessions	Supported
Health Checks	HTTP/HTTPS path-based
Cross-Zone LB	Enabled by default (free)

ALB Architecture



ALB Core Components

Component	Description
Listener	Checks incoming requests on a specific port
Listener Rule	Defines how traffic should be routed
Target Group	Group of backend servers/targets
Health Check	Monitors target health status

ALB Listener Rules — One-Line Definitions

Rule Type	One-Line Definition
Path-Based Routing	Routes traffic based on URL path
Host-Based Routing	Routes traffic based on domain/hostname
Header-Based Routing	Routes traffic based on HTTP headers
Query String Routing	Routes traffic using query parameters
HTTP Method Routing	Routes based on GET, POST, PUT, DELETE methods
Source IP Routing	Routes traffic based on client IP range

ALB Routing Examples

1. Path-Based Routing

```
example.com/api/*    —> API Servers
example.com/images/* —> Image Servers
example.com/admin/*  —> Admin Servers
```

2. Host-Based Routing

```
api.example.com    —> API Target Group
www.example.com     —> Web Target Group
admin.example.com   —> Admin Target Group
```

3. Header-Based Routing

```
Header: Device=Mobile —> Mobile Backend
Header: Device=Web    —> Web Backend
```

4. Query String Routing

```
?platform=mobile    —> Mobile Servers
?version=v2          —> Version 2 Backend
```

5. HTTP Method Routing

```
GET Requests    —> Read Servers
POST Requests    —> Write Servers
```

6. Source IP Routing

```
Corporate IP Range —> Internal Application
Public Users       —> Public Backend
```



3. Internet-Facing ALB vs Internal ALB

 Internet-Facing ALB

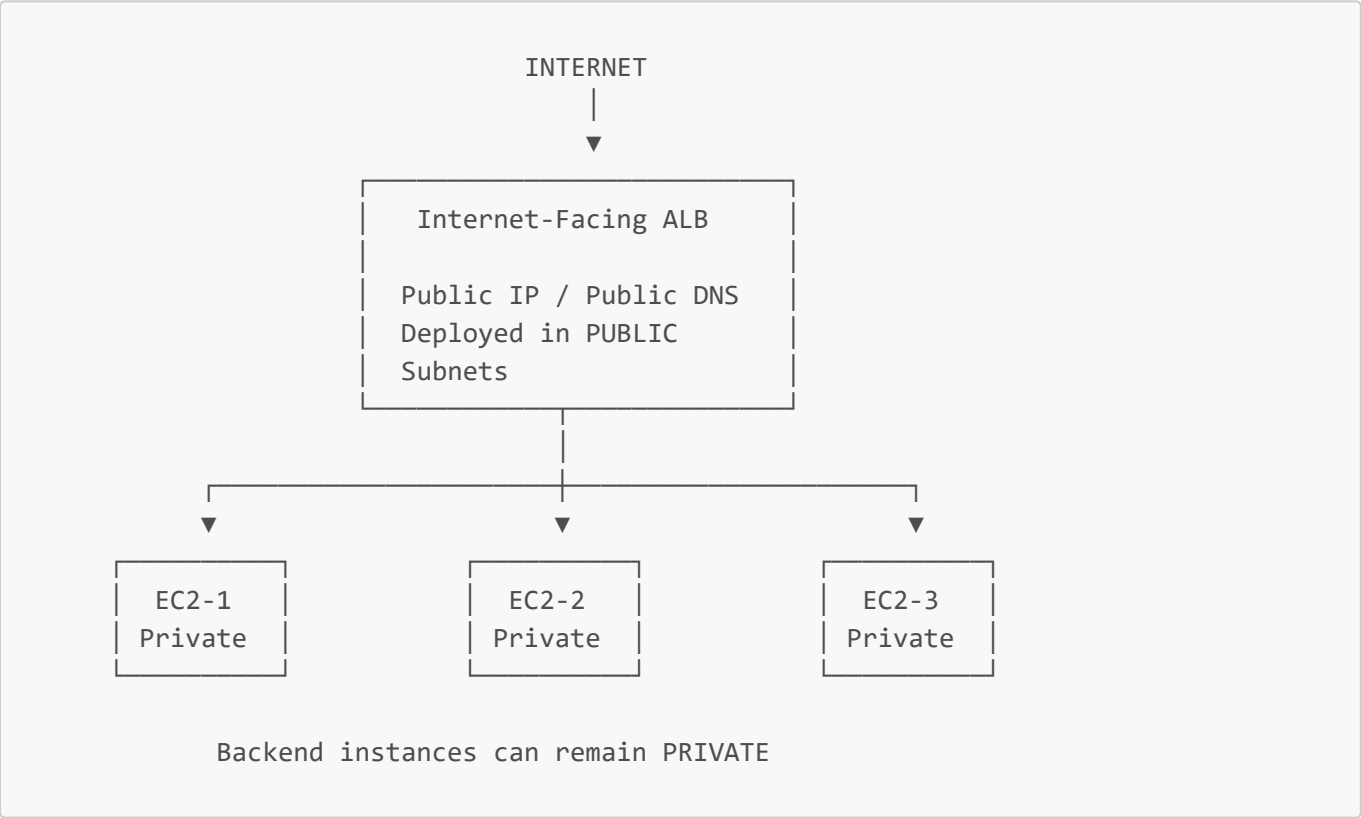
Definition

An **Internet-Facing ALB** has:

- Public IP addresses
- Public DNS name
- Internet accessibility

It receives traffic directly from the internet.

 Architecture



 Key Points

Feature	Details
DNS Name	Resolves to Public IPs
ALB Placement	Public Subnets
Targets	Usually Private Subnets

Feature	Details
Security Group	Allow 80/443 from 0.0.0.0/0
Accessed By	Public Internet Users
Use Cases	Websites, Public APIs

Internal ALB

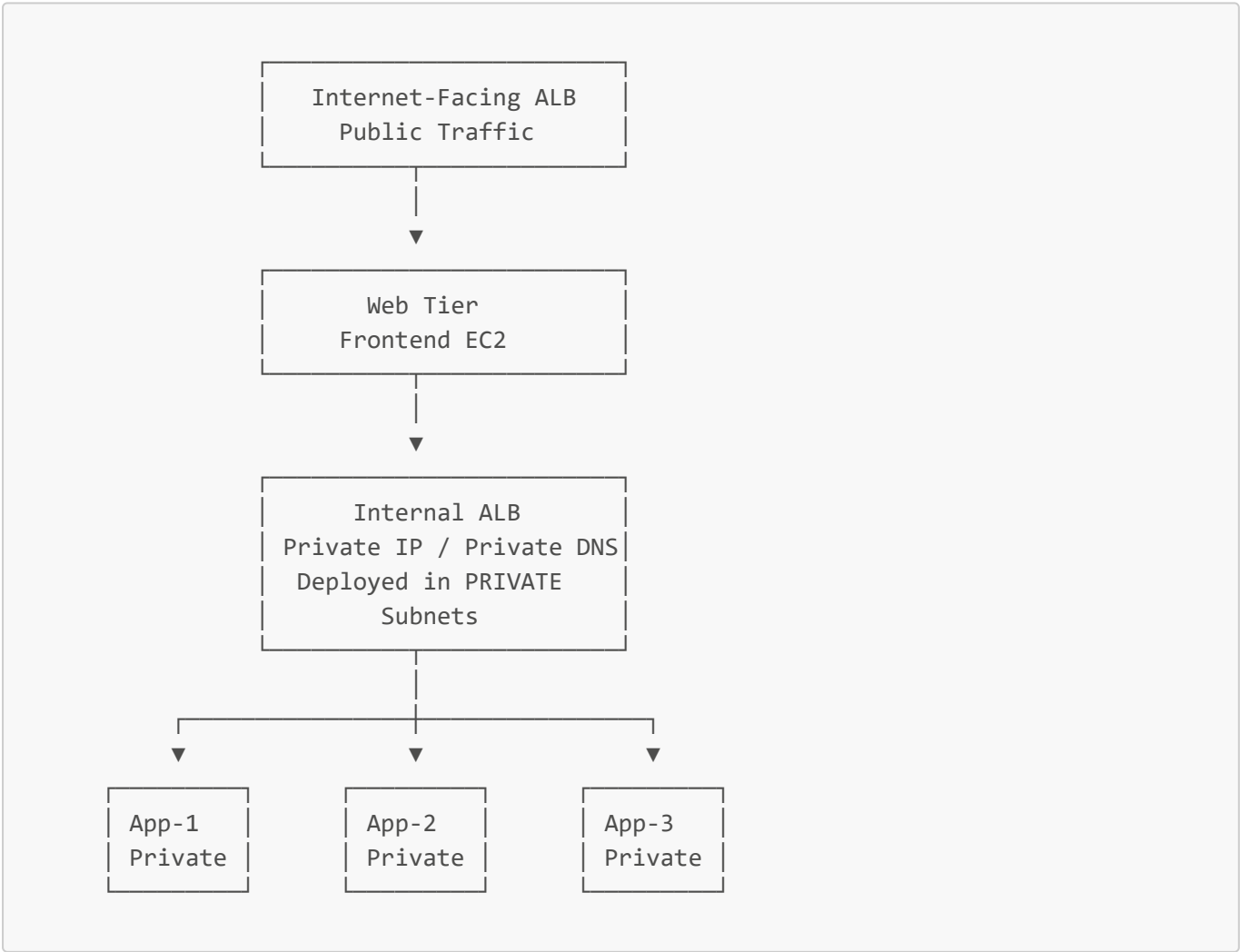
Definition

An **Internal ALB** has:

- Private IP addresses only
- Private DNS name
- No internet accessibility

Used for internal communication inside a VPC.

Architecture





Key Points

Feature	Details
DNS Name	Resolves to Private IPs
Placement	Private Subnets
Access	Only inside VPC
Security Group	Allow VPC CIDR or specific SGs
Use Cases	Microservices, Internal APIs



Internet-Facing ALB vs Internal ALB

Feature	Internet-Facing ALB	Internal ALB
IP Type	Public IP	Private IP
DNS Resolution	Public DNS	Private DNS
Subnets	Public	Private
Accessible From	Internet + VPC	VPC Only
Security Group	0.0.0.0/0	VPC CIDR / Specific SG
Position	Edge Entry Point	Between Application Tiers
Example	www.amazon.com	order-service.internal



4. Network Load Balancer (NLB) — Layer 4

Definition

A **Network Load Balancer (NLB)** operates at **OSI Layer 4 (Transport Layer)**.

It routes traffic based on:

- IP Address
- TCP/UDP Port

It does NOT inspect request content.

Designed for:

- Ultra High Performance
- Very Low Latency

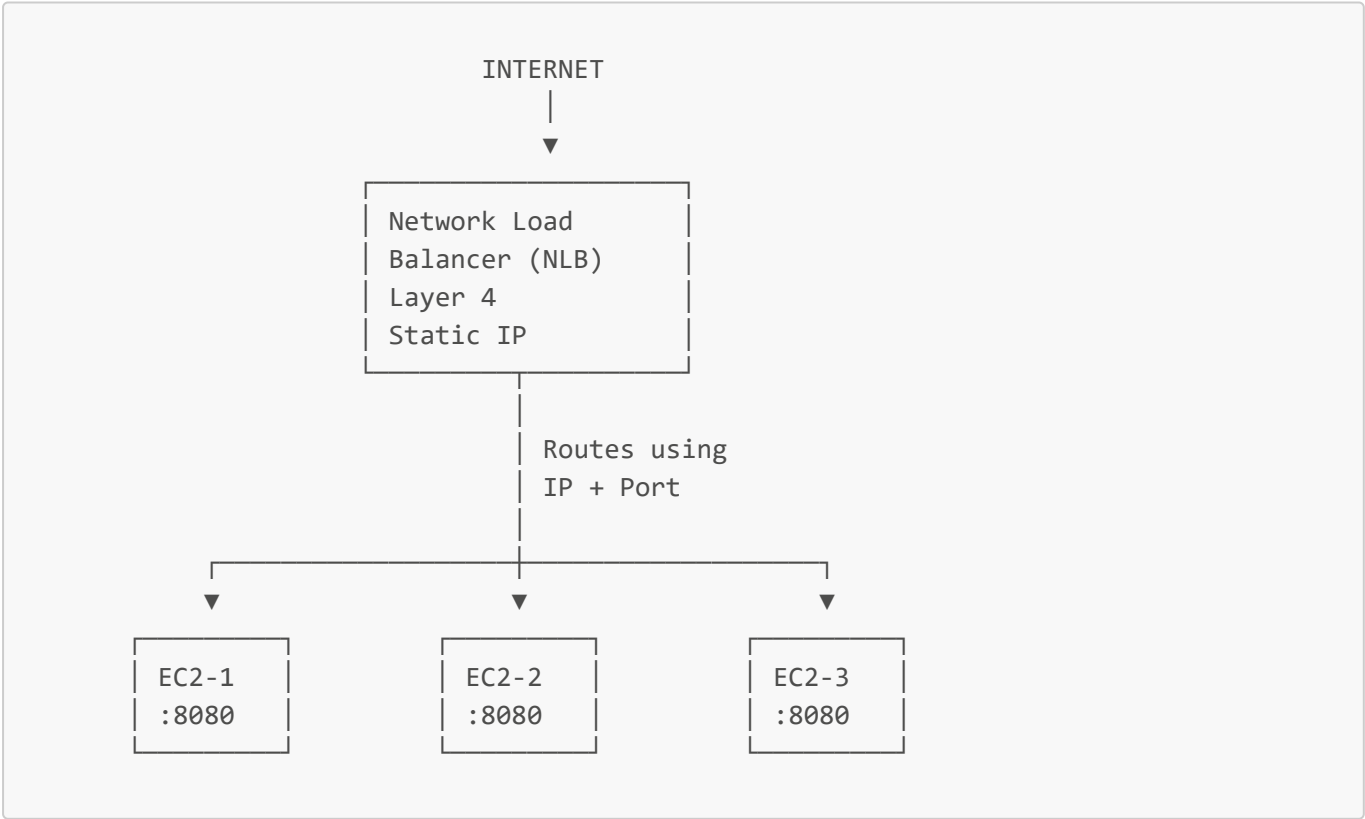
- Millions of requests per second

◆ NLB Key Characteristics

Feature	Description
Protocol Support	TCP, UDP, TLS
OSI Layer	Layer 4
Routing Type	IP + Port Based
Performance	Millions of requests/sec
Latency	Microseconds
Static IP	Supported
Elastic IP	Supported
Preserve Client IP	Yes
SSL Termination	Supported
Health Checks	TCP/HTTP/HTTPS



NLB Architecture



Why Static IP in NLB Matters

ALB

```
my-alb-123.elb.amazonaws.com
→ Dynamic IPs
→ IPs can change anytime
→ Use DNS name only
```

NLB

```
52.10.20.30
→ Static IP
→ Never changes
→ Can attach Elastic IP
→ Easy firewall whitelisting
```

5. ALB vs NLB — Detailed Comparison

Feature	ALB	NLB
OSI Layer	Layer 7	Layer 4
Protocols	HTTP, HTTPS, gRPC, WebSocket	TCP, UDP, TLS
Routing	Content-Based	IP + Port
Performance	High	Extreme
Latency	Milliseconds	Microseconds
Static IP	✗ No	✓ Yes
Elastic IP	✗ No	✓ Yes
Preserve Client IP	Via X-Forwarded-For	Native
SSL Termination	Supported	Supported
Sticky Sessions	Cookie-Based	Source IP Based
Lambda Support	✓ Yes	✗ No
ALB as Target	✗	✓
Cross-Zone LB	Enabled by Default	Disabled by Default

Feature	ALB	NLB
Security Groups	Supported	Not Supported

Important NLB Security Note

NLB does NOT have Security Groups.

So the backend EC2 instance sees the **real client IP**.

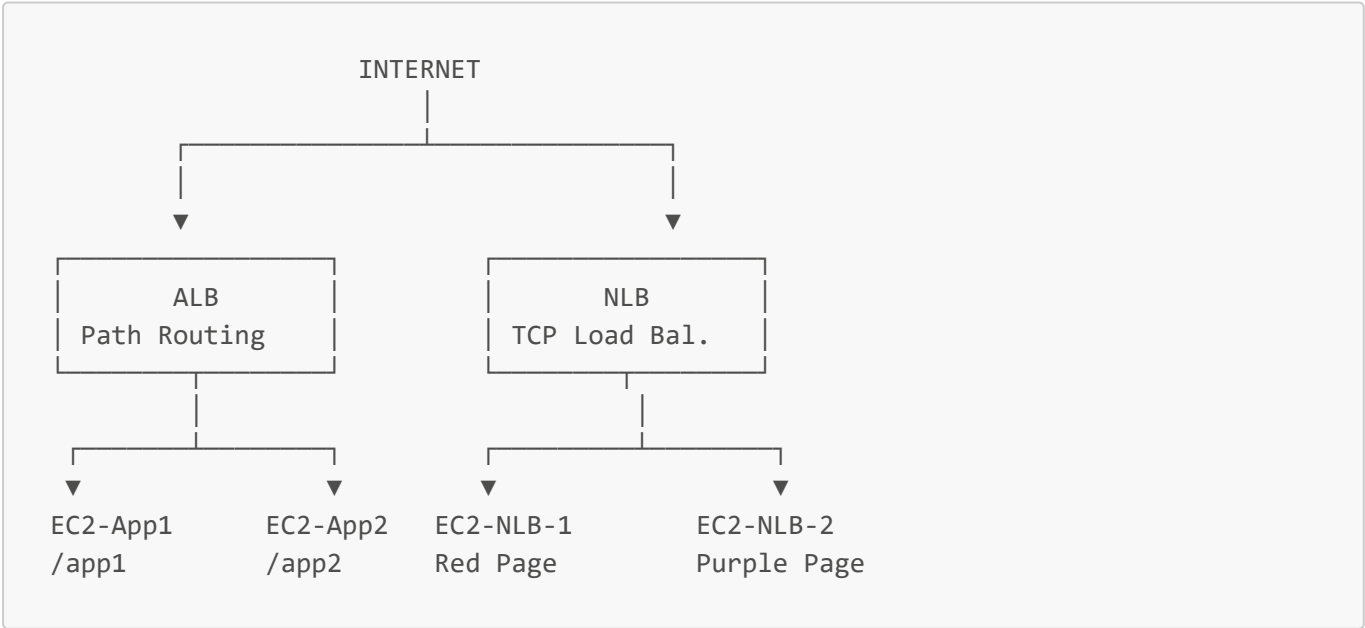
- That means:
- EC2 Security Group must allow client IP ranges directly
 - Traffic does NOT appear from NLB IPs

AWS Load Balancer Practice Tasks (ALB + NLB)

Tasks Overview

In this the following tasks were completed:

Architecture Used



Task 1 — Launch EC2-App1 Instance

Objective

Create an EC2 instance that serves the `/app1` application.

Configuration

Setting	Value
Name	EC2-App1
AMI	Ubuntu
Instance Type	t2.micro
VPC	LB-Practice-VPC
Subnet	Public-Subnet-1
Public IP	Enabled
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/app1

cat <<EOF > /var/www/html/app1/index.html
<html>
<body style='background-color:#4CAF50; text-align:center;'>
<h1 style='color:white; margin-top:200px;'>
Welcome to APP 1 ●
</h1>

<h2 style='color:white;'>
This is EC2 - App1 Server
</h2>

<h3 style='color:white;'>
Path: /app1
</h3>
</body>
</html>
EOF
```

Task 2 — Launch EC2-App2 Instance

Objective

Create another EC2 instance that serves the `/app2` application.

Configuration

Setting	Value
Name	EC2-App2
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/app2

cat <<EOF > /var/www/html/app2/index.html
<html>
<body style='background-color:#2196F3; text-align:center;'>
<h1 style='color:white; margin-top:200px;'>
Welcome to APP 2 ●
</h1>

<h2 style='color:white;'>
This is EC2 - App2 Server
</h2>

<h3 style='color:white;'>
Path: /app2
</h3>
</body>
```

```
</html>
EOF
```

Task 3 — Launch EC2-NLB-1 Instance

Objective

Create backend server 1 for Network Load Balancer testing.

Configuration

Setting	Value
Name	EC2-NLB-1
AMI	Ubuntu
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#FF5722; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
NLB Server 1 ●
</h1>

<h2 style='color:white;'>
This request was handled by NLB-Server-1
</h2>

</body>
</html>
EOF
```



Task 4 — Launch EC2-NLB-2 Instance

Objective

Create backend server 2 for Network Load Balancer testing.

Configuration

Setting	Value
Name	EC2-NLB-2
AMI	Ubuntu
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#9C27B0; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
NLB Server 2 ●
</h1>

<h2 style='color:white;'>
This request was handled by NLB-Server-2
</h2>

</body>
</html>
EOF
```



Task 5 — Create Target Group for App1

Objective

Create a target group for the App1 server.

Configuration

Setting	Value
Target Group Name	TG-App1
Protocol	HTTP
Port	80
Target Type	Instances
Health Check Path	/app1/index.html

Registered Target

EC2-App1



Task 6 — Create Target Group for App2

Objective

Create a target group for the App2 server.

Configuration

Setting	Value
Target Group Name	TG-App2
Protocol	HTTP
Port	80
Target Type	Instances
Health Check Path	/app2/index.html

Registered Target

EC2-App2

Task 7 — Create Application Load Balancer (ALB)

Objective

Create an Internet-Facing ALB for path-based routing.

Configuration

Setting	Value
Name	My-ALB
Type	Application Load Balancer
Scheme	Internet-Facing
Protocol	HTTP
Port	80
VPC	LB-Practice-VPC
Subnets	Public-Subnet-1 & Public-Subnet-2
Security Group	LB-SG

Default Action

Forward traffic to TG-App1

Task 8 — Configure Path-Based Routing

Objective

Route requests to different target groups based on URL paths.

Rule 1 — App1 Routing

Setting	Value
Condition Type	Path
Path Value	/app1*
Action	Forward to TG-App1
Priority	1

◆ Rule 2 — App2 Routing

Setting	Value
Condition Type	Path
Path Value	/app2*
Action	Forward to TG-App2
Priority	2

📌 Task 9 — Test ALB Path-Based Routing

Objective

Verify that ALB routes traffic correctly.

Test URLs

```
http://ALB-DNS/app1
```

Expected Result

```
Green APP1 page ●
```

```
http://ALB-DNS/app2
```

Expected Result

Task 10 — Create Target Group for NLB

Objective

Create target group for Network Load Balancer.

Configuration

Setting	Value
Name	TG-NLB
Protocol	TCP
Port	80
Target Type	Instances
Health Check	TCP

Registered Targets

EC2-NLB-1
EC2-NLB-2

Task 11 — Create Network Load Balancer (NLB)

Objective

Create an Internet-Facing NLB.

Configuration

Setting	Value
Name	My-NLB
Type	Network Load Balancer
Scheme	Internet-Facing

Setting	Value
Protocol	TCP
Port	80
Subnets	Public-Subnet-1 & Public-Subnet-2

Default Action

Forward traffic to TG-NLB

 Task 12 — Test NLB

Objective

Verify NLB distributes traffic between servers.

Test URL

http://NLB-DNS/

Expected Output

Sometimes:

NLB Server 1 

Sometimes:

NLB Server 2 

 Task 13 — Configure NLB in Front of ALB

Objective

Configure a **Network Load Balancer (NLB)** to forward traffic to an **Application Load Balancer (ALB)** using the special:

Application Load Balancer Target Type

This combines:

- NLB → Static IP + High Performance
- ALB → Smart Layer 7 Routing

What is "Application Load Balancer" Target Type?

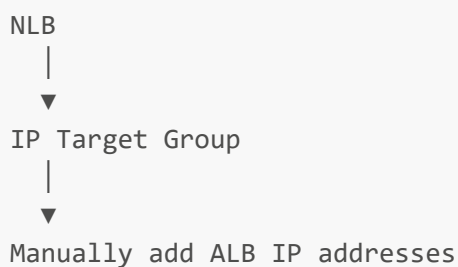
Definition

The **Application Load Balancer Target Type** is a special target type available in NLB Target Groups where the NLB can directly forward traffic to an ALB.

AWS automatically manages the ALB IP addresses internally.

Old Method vs New Method

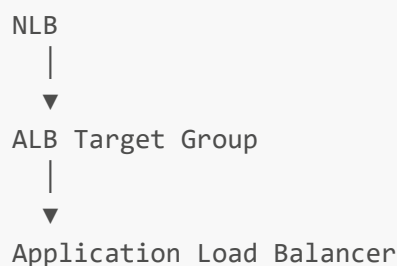
Old Method (Complex)



Problems

- ALB IPs change frequently
- Manual updates required
- Needed Lambda automation
- Complex architecture

New Method (Easy)



Benefits

- Direct ALB reference
- AWS manages IP changes
- Simple setup
- Production-ready architecture

Step 1 — Create Target Group with ALB Type

Navigation

EC2 → Target Groups → Create Target Group

Configuration

Setting	Value
Target Type	Application Load Balancer
Name	TG-NLB-to-ALB
Protocol	TCP
Port	80
VPC	LB-Practice-VPC

◆ Health Check Configuration

Setting	Value
Protocol	HTTP
Path	/app1/index.html

◆ Register Target

Select:

My-ALB

Port:

80

Click:

Create Target Group



Task 14 — Update NLB to Forward Traffic to ALB



Objective

Modify the existing NLB listener so traffic flows:

NLB → ALB → Target Groups → EC2 Instances



Step 2 — Update Existing NLB

Navigation

EC2 → Load Balancers → My-NLB

Steps

1. Go to:

Listeners Tab

2. Select:

TCP : 80 Listener

3. Click:

Actions → Edit Listener

4. Change Default Action:

Forward to: TG-NLB-to-ALB

5. Save Changes ☒



Step 3 — Test the Architecture

Test URL

http://NLB-DNS/app1

Expected Output

GREEN APP1 Page 

Test URL

http://NLB-DNS/app2

Expected Output



Complete Request Flow

Step 1:

User sends request to NLB DNS



Step 2:

NLB receives TCP traffic



Step 3:

NLB forwards traffic to ALB
(AWS internally manages ALB IPs)



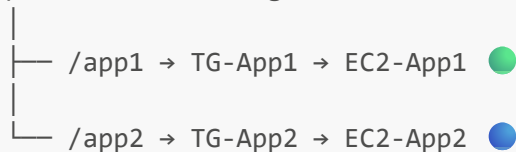
Step 4:

ALB reads HTTP request content



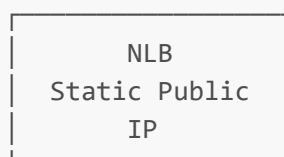
Step 5:

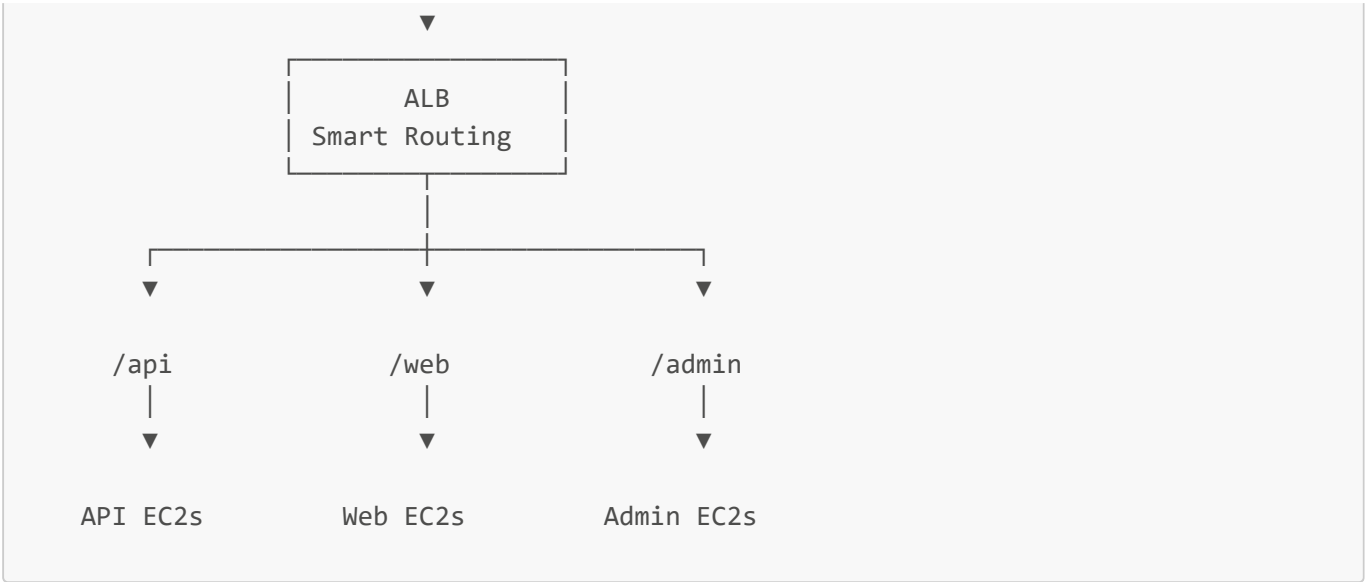
ALB applies path-based routing



Real World Architecture

INTERNET





Why This Architecture is Powerful

NLB Feature	Benefit
Static IP	Easy firewall whitelisting
High Performance	Handles millions of requests
Ultra Low Latency	Fast traffic forwarding

ALB Feature	Benefit
Path-Based Routing	Smart traffic distribution
Host-Based Routing	Domain-based routing
HTTP Awareness	Understands web requests

Real World Use Cases

This architecture is commonly used in:

- Banking Applications
- Government Systems
- Enterprise SaaS Platforms
- Healthcare Applications
- B2B APIs
- Security-Sensitive Systems

Why Companies Use NLB in Front of ALB

Because NLB provides:

- ✓ Static IP
- ✓ Elastic IP support
- ✓ IP Whitelisting
- ✓ High throughput
- ✓ Low latency

And ALB provides:

- ✓ Smart Layer 7 routing
- ✓ Path-based routing
- ✓ Host-based routing
- ✓ Header-based routing
- ✓ WebSocket/gRPC support

✓ Successfully implemented:

NLB → ALB → EC2 Architecture

with:

- Static Public Entry Point
- Advanced Routing
- High Availability
- Scalable Infrastructure
- Production-Level AWS Design
- Application Load Balancer (ALB)
- Path-Based Routing
- Network Load Balancer (NLB)
- Multi-Server Traffic Distribution
- Apache Web Hosting using User Data

AWS ALB Routing Methods Practice Tasks



Task 1 — Launch EC2-App1 (Host-Based Routing)

Objective

Server for:

app1.wellnest-project.online

Configuration

Setting	Value
Name	EC2-App1
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#4CAF50; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
● APP 1 Server
</h1>

<h2 style='color:white;'>
Host: app1.wellnest-project.online
</h2>

</body>
</html>
EOF
```



Task 2 — Launch EC2-App2 (Host-Based Routing)

Objective

Server for:

```
app2.wellnest-project.online
```

Configuration

Setting	Value
Name	EC2-App2
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#2196F3; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
● APP 2 Server
</h1>

<h2 style='color:white;'>
Host: app2.wellnest-project.online
</h2>

</body>
</html>
EOF
```

Task 3 — Launch EC2-API (Path-Based Routing)

Objective

Server for:

```
/api
```

Configuration

Setting	Value
Name	EC2-API
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/api

cat <<EOF > /var/www/html/api/index.html
<html>
<body style='background-color:#FF9800; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
● API Server
</h1>

<h2 style='color:white;'>
Path: /api
</h2>

</body>
```

```
</html>
EOF
```

Task 4 — Launch EC2-Web (Path-Based Routing)

Objective

Server for:

```
/web
```

Configuration

Setting	Value
Name	EC2-Web
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/web

cat <<EOF > /var/www/html/web/index.html
<html>
<body style='background-color:#9C27B0; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
  ● Web Server
</h1>

<h2 style='color:white;'>
```

```
Path: /web
</h2>

</body>
</html>
EOF
```

Task 5 — Launch EC2-Mobile (Query String Routing)

Objective

Server for:

```
?platform=mobile
```

Configuration

Setting	Value
Name	EC2-Mobile
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#E91E63; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
```

```
📱 Mobile Server
</h1>

<h2 style='color:white;'>
You are on Mobile Platform!
</h2>

<h3 style='color:white;'>
Routed via Query String
</h3>

</body>
</html>
EOF
```

Task 6 — Launch EC2-Desktop (Query String Routing)

Objective

Server for:

```
?platform=desktop
```

Configuration

Setting	Value
Name	EC2-Desktop
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y
```



```
systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#607D8B; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
 Desktop Server
</h1>

<h2 style='color:white;'>
You are on Desktop Platform!
</h2>

<h3 style='color:white;'>
Routed via Query String
</h3>

</body>
</html>
EOF
```

Task 7 — Create Target Groups

Target Groups Created

Target Group	Purpose
TG-App1	Host-Based Routing
TG-App2	Host-Based Routing
TG-API	Path-Based Routing
TG-Web	Path-Based Routing
TG-Mobile	Query String Routing
TG-Desktop	Query String Routing

Task 8 — Create Application Load Balancer

Configuration

Setting	Value
Name	My-ALB

Setting	Value
Type	Internet-Facing
Protocol	HTTP
Port	80
Security Group	LB-SG

Default Action

Forward to TG-Desktop

Task 9 — Configure Route 53 DNS Records

DNS Records Created

Record Name	Points To
wellnest-project.online	My-ALB
app1.wellnest-project.online	My-ALB
app2.wellnest-project.online	My-ALB

Task 10 — Configure Host-Based Routing Rules

Rule 1 — App1

Setting	Value
Host Header	app1.wellnest-project.online
Action	Forward to TG-App1
Priority	1

Rule 2 — App2

Setting	Value
Host Header	app2.wellnest-project.online

Setting	Value
Action	Forward to TG-App2
Priority	2

Task 11 — Configure Path-Based Routing Rules

◆ API Rule

Setting	Value
Path	/api*
Action	Forward to TG-API
Priority	3

◆ Web Rule

Setting	Value
Path	/web*
Action	Forward to TG-Web
Priority	4

Task 12 — Configure Query String Routing Rules

◆ Mobile Query Rule

Setting	Value
Query Key	platform
Query Value	mobile
Action	Forward to TG-Mobile
Priority	7

◆ Desktop Query Rule

Setting	Value
Query Key	platform
Query Value	desktop
Action	Forward to TG-Desktop
Priority	8



Task 13 — Launch EC2-GET (HTTP Method Routing)

Objective

Handle GET requests.

Configuration

Setting	Value
Name	EC2-GET
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#00BCD4; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
📖 GET Server - Read Operations
</h1>

<h2 style='color:white;'>
You sent a GET Request!
```

```
</h2>

<h3 style='color:white;'>
This server handles READ operations only
</h3>

</body>
</html>
EOF
```

Task 14 — Launch EC2-POST (HTTP Method Routing)

Objective

Handle POST requests.

Configuration

Setting	Value
Name	EC2-POST
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#F44336; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
 POST Server - Write Operations
```

```
</h1>

<h2 style='color:white;'>
You sent a POST Request!
</h2>

<h3 style='color:white;'>
This server handles WRITE operations only
</h3>

</body>
</html>
EOF
```

Task 15 — Create Target Groups for HTTP Method Routing

Target Group	Purpose
TG-GET	GET Requests
TG-POST	POST Requests

Task 16 — Configure HTTP Method Routing Rules

GET Rule

Setting	Value
HTTP Method	GET
Action	Forward to TG-GET
Priority	9

POST Rule

Setting	Value
HTTP Method	POST
Action	Forward to TG-POST
Priority	10

Task 17 — Test Host-Based Routing

Test URLs

```
http://app1.wellnest-project.online
```

Expected:

```
GREEN APP1 Page 
```

```
http://app2.wellnest-project.online
```

Expected:

```
BLUE APP2 Page 
```

Task 18 — Test Path-Based Routing

Test URLs

```
http://wellnest-project.online/api
```

Expected:

```
ORANGE API Page 
```

```
http://wellnest-project.online/web
```

Expected:

PURPLE WEB Page 

Task 19 — Test Query String Routing

Test URLs

```
http://wellnest-project.online?platform=mobile
```

Expected:

PINK Mobile Page 

```
http://wellnest-project.online?platform=desktop
```

Expected:

GREY Desktop Page 

Task 20 — Test HTTP Method Routing

Test GET Request

```
curl -X GET http://wellnest-project.online
```

Expected:

GET Server Page 

Test POST Request


```
curl -X POST http://wellnest-project.online
```

Expected:

```
POST Server Page 
```



Routing Methods Practiced

Routing Type	Status
Host-Based Routing	 Completed
Path-Based Routing	 Completed
Query String Routing	 Completed
HTTP Method Routing	 Completed

Successfully implemented:

-  Host-Based Routing
-  Path-Based Routing
-  Query String Routing
-  HTTP Method Routing
-  Route 53 + ALB Integration
-  Multi-Target Group Architecture



DNS Fundamentals & AWS Route 53 — Study Notes



Table of Contents

- [1. What is DNS?](#)
- [2. DNS Resolution Journey](#)
- [3. Core DNS Terminology](#)
- [4. DNS Infrastructure Components](#)
- [5. Domain Registrars & DNS Providers](#)
- [6. AWS Route 53 Overview](#)

7. [Route 53 Building Blocks](#)
8. [DNS Record Types Explained](#)
9. [Traffic Routing Strategies](#)
10. [Health Monitoring in Route 53](#)
11. [Hands-On Labs](#)
12. [Summary & Cheat Sheet](#)

What is DNS?

The **Domain Name System (DNS)** is essentially a massive, globally distributed directory that converts website names into numerical IP addresses that computers understand.

```
User types → www.amazon.com
DNS returns → 205.251.242.103
Browser connects → Web page loads
```

Think of it this way — your smartphone contacts list stores names instead of making you memorize phone numbers. DNS does the exact same thing for the internet. Instead of remembering **172.217.18.36**, you just type **google.com**.

Why Does DNS Matter?

Problem Without DNS	Solution With DNS
Must memorize IPs like 54.239.28.85	Just type amazon.com
Changing server IPs breaks access	DNS record updates are transparent
No scalability for millions of sites	Distributed hierarchy handles global scale

DNS Resolution Journey

Every time you visit a website, a behind-the-scenes lookup process happens in milliseconds. Here's a breakdown of that process:

```
Step 1: You type "www.netflix.com" in browser
      ↓
Step 2: Browser Cache – Already visited before?
      ↓ (No)
Step 3: OS Cache – Stored in system?
      ↓ (No)
Step 4: Recursive Resolver (ISP or 8.8.8.8)
      ↓
Step 5: Root Server – "Where is .com?"
      ↓
Step 6: TLD Server (.com) – "Who manages netflix.com?"
      ↓
```

```
Step 7: Authoritative Server – "netflix.com = 54.74.12.1"
      ↓
Step 8: Browser connects → Page loads ✅
Step 9: Result cached based on TTL
```

🏠 Who Does What?

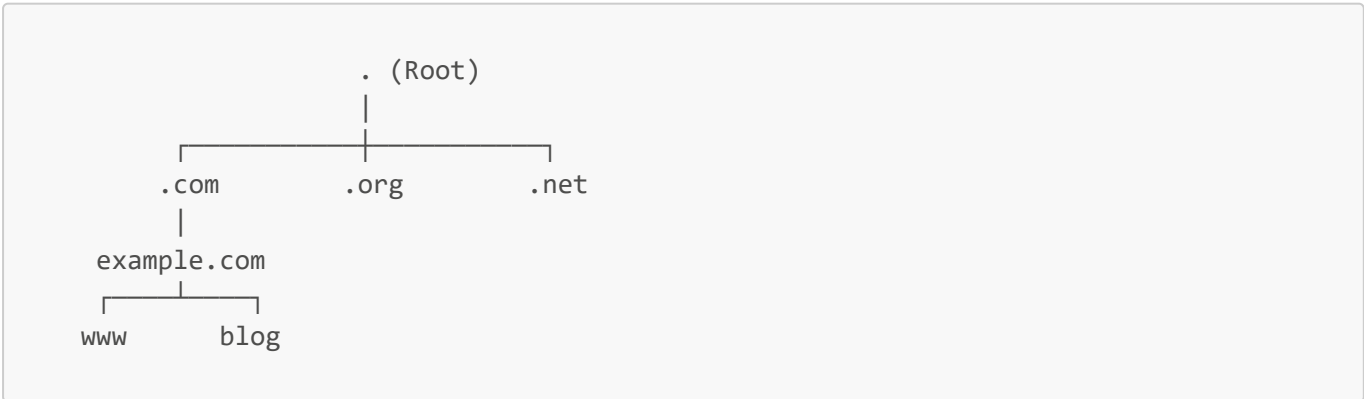
DNS Component	Real-World Analogy	Function
Recursive Resolver	A librarian who does the research for you	Performs full lookup on behalf of client
Root Name Server	Library's main index	Points to correct TLD server
TLD Server	A section divider in the library	Manages <code>.com</code> , <code>.org</code> , <code>.net</code> etc.
Authoritative Server	The actual book you needed	Holds and returns final DNS records
DNS Cache	Your personal bookmarks	Stores recent lookups for speed

📖 Core DNS Terminology

Term	Plain-English Meaning	Example
Domain Name	The website address users type	<code>shopify.com</code>
IP Address	Numeric address of the actual server	<code>23.227.38.65</code>
TTL (Time To Live)	How long a DNS record stays cached	<code>300 seconds</code>
FQDN	The full, complete domain name including the dot	<code>www.shopify.com.</code>
TLD	The ending part of a domain	<code>.com</code> , <code>.in</code> , <code>.org</code>
Zone File	A text file containing all DNS records	Records for <code>example.com</code>

🏛️ DNS Infrastructure Components

How the Hierarchy Looks



The DNS system is like a tree — queries start at the root and travel down branches until they find the answer.

Domain Registrars & DNS Providers

The Difference Explained

Domain Registrar = Who owns the domain name
DNS Hosting Provider = Who answers DNS queries for that domain

These can be the same company or different companies — your choice.

Real Example:

- Buy domain **mystore.com** on **Namecheap**
- Host DNS records on **AWS Route 53**
- Point Namecheap nameservers → Route 53

Popular Options

Service	Category	Strength
GoDaddy	Registrar + DNS	Widely used, beginner-friendly
Namecheap	Registrar + DNS	Budget-friendly, includes privacy protection
Cloudflare	DNS + CDN	Extremely fast, DDoS protection included
AWS Route 53	Registrar + DNS	Deep AWS integration, advanced features

AWS Route 53 Overview

What Exactly is Route 53?

Route 53 is Amazon's **fully managed, highly available DNS and traffic routing service**. It's authoritative, meaning you have full control to create, edit, and delete DNS records directly.

Route 53 handles three things:

1. Domain Registration
2. DNS Record Management
3. Traffic Routing + Health Checks

Where Does the Name "Route 53" Come From?

"Route" → It routes internet traffic to destinations
"53" → DNS communicates over Port 53

Protocol	Port Number
HTTP	80
HTTPS	443
FTP	21
DNS	53 ←

What Makes Route 53 Special?

Capability	Description
100% Uptime SLA	AWS guarantees full availability
Global Anycast Network	DNS servers spread worldwide
Intelligent Routing	8 different routing strategies
Health Monitoring	Automatically detects and avoids broken servers
AWS Native	Works seamlessly with EC2, ALB, CloudFront, S3

 Route 53 Building Blocks

 Hosted Zones

A Hosted Zone is like a folder that stores all DNS records for one domain. When you create a hosted zone for `example.com`, everything DNS-related for that domain lives inside it.

Hosted Zone: `example.com`

- └─ A Record → `www.example.com` → `54.1.2.3`
- └─ MX Record → `mail.example.com` → Google Mail
- └─ TXT Record → Verification codes
- └─ CNAME Record → `shop.example.com` → `shopify.com`

Two Types of Hosted Zones

Type	Accessible From	Use Case
 Public Hosted Zone	Anywhere on the internet	Public-facing websites
 Private Hosted Zone	Only inside a specific VPC	Internal apps, databases

When you create a hosted zone, Route 53 automatically generates an **NS record** and an **SOA record** for you.

Name Servers (NS)

Name Servers are the servers that actually hold and serve your DNS records. Route 53 assigns **4 name servers** to every hosted zone for redundancy.

Sample Route 53 Name Servers:

```
ns-245.awsdns-30.com
ns-1532.awsdns-07.co.uk
ns-831.awsdns-40.net
ns-1267.awsdns-30.org
```

Having 4 different name servers means even if one fails, the others keep answering queries.

DNS Delegation

Delegation is the process of telling the world "Route 53 is in charge of DNS for my domain."

```
graph TD
    A[Your Domain (bought on GoDaddy)] --> B[Login to GoDaddy → Change Nameservers]
    B --> C[Paste Route 53 NS Records]
    C --> D[DNS queries now handled by Route 53 ✓]
```

Without delegation, Route 53 has the records but nobody knows to ask Route 53 for them.

SOA Record (Start of Authority)

The SOA record is the administrative ID card of your DNS zone. It tells other DNS servers basic facts about your zone.

Contains:

- Which server is the primary name server
- Admin contact email
- A serial number (zone version tracker)
- Timing settings for zone refreshes

Route 53 manages SOA records automatically — you rarely need to touch this.

🕒 TTL (Time To Live)

TTL is a number (in seconds) that tells DNS resolvers how long to remember a record before checking again.

TTL = 300 → Cache for 5 minutes → Faster to change, more queries
TTL = 86400 → Cache for 24 hours → Fewer queries, slow to update

Scenario	Recommended TTL
Making changes soon	Low (60–300 sec)
Stable, no changes planned	High (3600–86400 sec)
Before a migration	Drop to low TTL first

📄 DNS Record Types Explained

📖 Visual Overview

DNS Record Toolkit

- A → domain name → IPv4 address
- AAAA → domain name → IPv6 address
- CNAME → domain name → another domain name
- MX → domain name → mail server
- TXT → domain name → plain text data
- NS → domain name → name server
- PTR → IP address → domain name
- SRV → service name → host + port
- ★ Alias → domain name → AWS resource

● A Record

The most basic and commonly used DNS record. Maps a domain name to an IPv4 address.

Record Type : A
Name : mywebsite.com
Value : 13.234.56.78
TTL : 300

Simple example: Typing `mywebsite.com` → A record → browser connects to `13.234.56.78`

🟦 AAAA Record

Exactly like an A record but for IPv6 addresses (the newer, longer IP format).

```
Record Type : AAAA
Name       : mywebsite.com
Value      : 2400:cb00:2048:1::c629:d7a2
TTL        : 300
```

🟡 CNAME Record

Creates an alias from one domain name to another domain name.

```
blog.mywebsite.com → mywebsite.com
shop.mywebsite.com → stores.shopify.com
```

⚠ Important Limitation:

- ❌ CANNOT use CNAME for the root domain
mywebsite.com → NOT allowed as a CNAME
- ✅ CAN use CNAME for subdomains
www.mywebsite.com → Allowed ✓

📧 MX Record

Tells email servers where to deliver emails for your domain.

```
Someone sends email to → hello@mywebsite.com
MX Record says         → Deliver to mail.google.com
Priority 1              → Try this server first
Priority 10             → Fallback server
```

Priority	Server	Role
1	aspmx.l.google.com	Primary mail server
5	alt1.aspmx.l.google.com	Secondary
10	alt2.aspmx.l.google.com	Tertiary

📄 TXT Record

Stores plain text data attached to your domain. Primarily used for verification and email security.

Use Case	What It Does
Domain Verification	Proves you own the domain to Google/Microsoft
SPF	Lists which servers can send email on your behalf
DKIM	Adds a digital signature to verify emails weren't tampered with
DMARC	Policy for handling emails that fail SPF/DKIM checks

```
Example SPF record:  
"v=spf1 include:_spf.google.com ~all"
```

NS Record

Declares which name servers are authoritative for your domain. Auto-created by Route 53.

PTR Record

The opposite of an A record — maps an IP address back to a domain name. Used in reverse DNS lookups.

```
Normal:  mywebsite.com    → 54.12.34.56  (A Record)  
Reverse: 54.12.34.56     → mywebsite.com (PTR Record)
```

Common uses: Email spam filtering, server logging, network debugging.

SRV Record

Specifies both the hostname AND port number for a specific service. Helps clients automatically discover services.

```
Format: Priority  Weight  Port  Target  
        10       20     5060  sip.mywebsite.com
```

Used in: VoIP, gaming servers, SIP, chat protocols (XMPP).

Alias Records

A Route 53 exclusive feature. Works like a CNAME but with superpowers — specifically designed for AWS services.

```
mywebsite.com → my-load-balancer-123.us-east-1.elb.amazonaws.com
```

Supported AWS targets:

- Application Load Balancer (ALB)
- CloudFront distributions
- API Gateway
- S3 static website buckets
- Global Accelerator

 CNAME vs Alias — Side by Side

Feature	CNAME Record	Alias Record
Works on root domain (<i>example.com</i>)	✗ No	✓ Yes
Extra DNS query needed	Yes	No (resolved internally)
AWS IP changes handled automatically	✗ No	✓ Yes
Cost for AWS resource queries	Charged	Free
Health check support	Limited	✓ Native
Works outside AWS	✓ Yes	✗ AWS targets only

Rule of thumb: Always prefer Alias records when pointing to AWS resources.

 Traffic Routing Strategies

Route 53 routing policies control **which answer DNS returns** when a user queries your domain. DNS does not route actual network packets — it only decides what IP or endpoint to tell the user about.

 All Policies at a Glance

Policy	Decision Factor	Health Checks	Best For
Simple	None (just returns value)	✗	Single-server basics
Weighted	Percentage split	✓	Gradual rollouts, A/B tests
Latency	Fastest response time	✓	Global performance
Failover	Primary or backup	✓ Required	Disaster recovery
Geolocation	User's country/region	✓	Localization
Geoproximity	Location + bias value	✓	Fine-tuned traffic shaping
Multi-Value	Multiple healthy IPs	✓	Basic distribution
IP-Based	Client IP / CIDR block	✓	ISP or enterprise routing

1 Simple Routing

The most straightforward policy — one domain, one destination.

```
user queries example.com
      ↓
Route 53 returns → 54.12.34.56
```

If you add multiple IPs, Route 53 returns all of them and the browser picks one randomly.

Limitation: No smart failover. If server goes down,
Route 53 still returns that IP ❌

Best for: Personal projects, single-server applications, testing environments.

2 Weighted Routing ⚖️

Distributes traffic across multiple servers using assigned weight values.

Traffic Split Example:

```
Server A (New Version) → Weight 10 → Gets 10% of users
Server B (Old Version) → Weight 90 → Gets 90% of users
```

The math:

$$\text{Server weight} \div \text{Total weight} = \% \text{ of traffic}$$

Weight 70 out of 100 total = 70% traffic

Weight 30 out of 100 total = 30% traffic

Weight Value	Behavior
Any positive number	Proportional traffic share
0	No traffic sent to this resource
All records = 0	Traffic distributed equally

Best for: Blue-green deployments, canary releases, gradual feature rollouts.

3 Latency Routing ⚡

Sends each user to whichever AWS region gives them the fastest response — not necessarily the closest geographically.

A user in Singapore queries example.com:

Route 53 checks:

Singapore → ap-southeast-1 latency: 8ms ← Winner ✓
US East → us-east-1 latency: 190ms
Ireland → eu-west-1 latency: 220ms

Result: User routed to Singapore region

Geography ≠ Latency. A user in Germany might get lower latency from a US server than a European one depending on network conditions.

Best for: Global applications where user experience and response speed matter.

4 Failover Routing 🛡️

Maintains a primary server and a standby backup. Automatically switches when primary becomes unhealthy.

Normal State:

User → Primary Server (Healthy ✓) → Serves traffic

Failure State:

User → Primary Server (Down ✗)
→ Route 53 detects via health check
→ Secondary Server (Standby) → Takes over ✓

Record Role	Configuration	When Used
Primary	Failover = Primary	Always (when healthy)
Secondary	Failover = Secondary	Only when primary fails

Requires: Active health checks on the primary record.

Best for: Disaster recovery setups, high-availability critical applications.

5 Geolocation Routing 📍

Routes traffic based on where the user physically is located.

User in India → Routed to Mumbai server
User in Germany → Routed to Frankfurt server

User in Brazil → Routed to Default server (no Brazil-specific record)

Supported location granularity:

- Entire continent (e.g., Europe, Asia)
- Specific country (e.g., India, USA)
- US States specifically

Always create a **Default** record to catch users from unspecified locations.

Best for: Serving localized content, regional pricing, media licensing restrictions, GDPR compliance.

6 Geoproximity Routing 📍

Similar to Geolocation but adds a **bias** — letting you artificially expand or shrink which users get routed to which region.

Without Bias:	With Positive Bias (+50) on US-East:
EU → Europe	EU users near the boundary → US-East
US → US-East	US → US-East (even larger share)

Bias Setting	Traffic Impact
+1 to +99	Pull more users toward this region
-1 to -99	Push users away from this region
0	Default (no adjustment)

Requires **Route 53 Traffic Flow** to configure. More complex but very powerful.

Best for: Situations where you want business logic to override pure geography.

7 Multi-Value Answer Routing 🔄

Returns multiple healthy IP addresses in response to a single query. The client (browser/application) picks one.

Query for example.com returns:

```

54.12.34.56 (Healthy ✓)
54.12.34.99 (Healthy ✓)
54.12.34.11 (Unhealthy ✗ – excluded automatically)
```

- Returns up to **8 healthy records** per query
- Each record can have its own health check
- Unhealthy records are automatically filtered out

This is **not** the same as a Load Balancer. It provides basic distribution but lacks advanced features like session persistence.

Best for: Simple distribution across multiple EC2 instances or servers.

8 IP-Based Routing 🌐

Routes traffic based on the client's source IP address or CIDR block.

```
Users from 203.0.113.0/24 → Server A (e.g., ISP-specific optimized server)
Users from 198.51.100.0/24 → Server B
Everyone else → Default server
```

Best for: Routing specific ISPs, enterprise networks, or branch offices to dedicated endpoints.

❤️ Health Monitoring in Route 53

Route 53 Health Checks continuously probe your endpoints and mark them as healthy or unhealthy — automatically influencing routing decisions.

Route 53 Health Checkers (15 globally distributed)

Send HTTP/HTTPS/TCP probe to your endpoint

```
graph TD
    A[Route 53 Health Checkers (15 globally distributed)] --> B[Send HTTP/HTTPS/TCP probe to your endpoint]
    B --> C[200 OK]
    B --> D[Timeout/Error]
    C --> E[Healthy]
    E --> F[✓]
    D --> G[Unhealthy]
    G --> H[✗]
```

Route 53 stops
sending traffic here

Three Types of Health Checks

◆ Type 1 — Endpoint Monitoring

Directly tests a URL, IP, or domain using HTTP, HTTPS, or TCP.

```
Configuration example:
Protocol   : HTTPS
Endpoint   : api.mywebsite.com
Port       : 443
Path       : /health
```

```
Interval   : 30 seconds
Threshold  : 3 failures = unhealthy
```

◆ Type 2 — Calculated Health Check

Combines results from multiple child health checks using logic operators.

```
Parent Check = Healthy IF:
  (Check-A AND Check-B) OR (Check-C)
```

Useful for: Treating a resource as healthy only when multiple conditions are met.

◆ Type 3 — CloudWatch Alarm Health Check

Uses an existing CloudWatch Alarm as the health signal instead of directly probing.

```
CloudWatch Alarm (CPU > 90% for 5 min)
    ↓
  ALARM state
    ↓
Route 53 marks endpoint as Unhealthy ❌
```

Essential for **private VPC resources** — Route 53 health checkers are public and cannot reach private IPs directly.

✅ What Counts as Healthy?

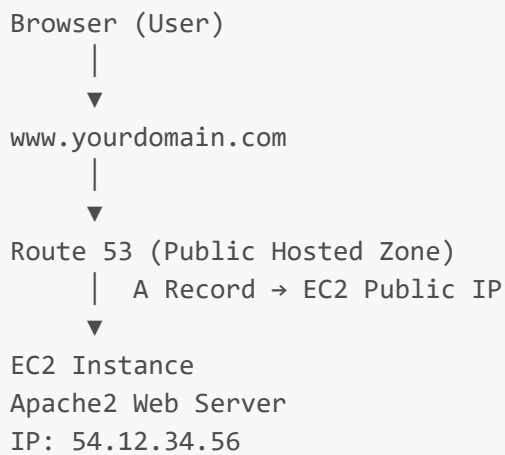
Response	Status
HTTP 2xx or 3xx	✅ Healthy
HTTP 4xx, 5xx	❌ Unhealthy
Connection timeout	❌ Unhealthy
Optional: Response body match	Must contain expected string

🔧 Hands-On Labs

🔧 Lab 1 — Host a Website with Simple Routing

Goal: Take an EC2 instance, install a web server, and make it accessible via a custom domain through Route 53.

Architecture Diagram



🔧 Step 1 — Get a Domain Name

Option A — Buy from Namecheap / GoDaddy:

- Purchase your preferred domain
- You'll update nameservers later to point to Route 53

Option B — Register directly in Route 53:

- AWS Console → Route 53 → Register Domain
- Hosted zone gets created automatically

🔧 Step 2 — Launch EC2 Instance

Configuration	Value
AMI	Ubuntu 22.04 LTS
Instance Type	t2.micro (free tier)
Auto-assign Public IP	Enable
Security Group — Inbound	SSH (port 22), HTTP (port 80)
Key Pair	Create or use existing

1. Go to EC2 → Launch Instance
2. Configure settings above
3. Launch and note down the **Public IPv4 address**

🔧 Step 3 — Set Up Web Server

SSH into the instance:


```
ssh -i your-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Install and start Apache2:

```
sudo apt update -y
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
```

Create a test webpage:

```
sudo bash -c 'cat > /var/www/html/index.html << EOF
<!DOCTYPE html>
<html>
  <head><title>My Route 53 Site</title></head>
  <body>
    <h1>🚀 Website Live via AWS Route 53!</h1>
    <p>DNS is working correctly.</p>
  </body>
</html>
EOF'
```

Quick verify — paste EC2 public IP in browser:

```
http://YOUR_EC2_PUBLIC_IP
```

You should see your webpage.

🔧 Step 4 — Create Route 53 Hosted Zone

1. AWS Console → Route 53 → Hosted Zones
2. Click **Create Hosted Zone**

Field	Value
Domain Name	yourdomain.com
Type	Public Hosted Zone

3. After creation, copy the **4 NS record values** shown

```
Example NS values:
ns-388.awsdns-48.com
```

```
ns-1412.awsdns-48.org
ns-763.awsdns-31.net
ns-1659.awsdns-15.co.uk
```

Step 5 — Point Domain to Route 53

For Namecheap:

- Dashboard → Domain List → Manage
- Nameservers section → Select **Custom DNS**
- Paste all 4 Route 53 NS values

For GoDaddy:

- My Products → DNS Settings
- Nameservers → Change → Select **I'll use my own nameservers**
- Paste all 4 Route 53 NS values

 DNS propagation can take anywhere from a few minutes to 48 hours globally.

Step 6 — Create the A Record

1. Route 53 → Your Hosted Zone → **Create Record**

Field	Value
Record Name	www (or leave blank for root)
Record Type	A
Routing Policy	Simple
Value	Your EC2 Public IP
TTL	300

2. Save the record

Step 7 — Test & Validate

Command line checks:

```
# Check what IP the domain resolves to
nslookup yourdomain.com

# Detailed DNS lookup
dig yourdomain.com
```

```
# Check from a specific DNS server
dig @8.8.8.8 yourdomain.com
```

Browser test:

```
http://yourdomain.com
```

✓ You should see your custom webpage served from EC2!

 Lab 2 — Weighted Routing (Traffic Splitting)

Goal: Split traffic between two servers — e.g., 80% to stable version, 20% to new version.

Setup — Two EC2 instances with different pages:

On Server 1:

```
echo "<h1>Server 1 – Stable Version</h1>" | sudo tee /var/www/html/index.html
```

On Server 2:

```
echo "<h1>Server 2 – New Version (Beta)</h1>" | sudo tee /var/www/html/index.html
```

Route 53 Records:

Record	IP	Weight	Set ID
example.com	Server1-IP	80	server-stable
example.com	Server2-IP	20	server-beta

Verify traffic split:

```
for i in {1..10}; do
  dig +short yourdomain.com
  sleep 1
done
```

You should see Server 1's IP appear ~8 times and Server 2's ~2 times.

 Lab 3 — Latency Routing

Goal: Route users to the AWS region that gives them the lowest latency.

Setup — EC2 instances in multiple regions:

Region	Location	Instance IP
ap-south-1	Mumbai	IP-Mumbai
us-east-1	N. Virginia	IP-Virginia
eu-west-1	Ireland	IP-Ireland

Route 53 Records:

For each region, create a record with:

Field	Value
Record Type	A
Routing Policy	Latency
Region	Select the EC2's AWS region
Value	That region's EC2 IP
Health Check	Attach one

Route 53 automatically routes each user to their lowest-latency region.

Lab 4 — Failover Routing

Goal: Keep a backup server ready to take over if the primary goes down.

Create a Health Check First

Route 53 → Health Checks → Create Health Check

Field	Value
Name	primary-server-check
Monitor	Endpoint
Protocol	HTTP
IP / Domain	Primary server IP
Port	80
Path	/health
Interval	30 seconds
Failure Threshold	3

Create Health Check Endpoint on Primary Server:

```
sudo mkdir -p /var/www/html/health
echo "OK" | sudo tee /var/www/html/health/index.html
```

Create Failover DNS Records

Primary Record:

Field	Value
Record Type	A
Routing Policy	Failover
Failover Type	Primary
Value	Primary server IP
Health Check	primary-server-check

Secondary Record:

Field	Value
Record Type	A
Routing Policy	Failover
Failover Type	Secondary
Value	Backup server IP

Test Failover

Simulate primary server failure:

```
# On primary server – stop Apache
sudo systemctl stop apache2
```

Wait ~90 seconds for health check failure threshold to trigger, then:

```
dig +short yourdomain.com
```

Should now return the **secondary server's IP** 

Restore primary:

```
sudo systemctl start apache2
```

Lab 5 — Geolocation Routing

Goal: Serve different content to users based on their country.

Setup servers in each target region and create records:

Record	Geolocation	IP	Description
example.com	India	India-Server-IP	Indian users
example.com	United States	US-Server-IP	US users
example.com	Default	Global-Server-IP	Everyone else

Without a Default record, users from unmapped locations get a "no response" error.

Lab 6 — Multi-Value Routing

Goal: Return multiple healthy IPs so clients can pick any available server.

Create separate A records for each server:

Field	Record 1	Record 2	Record 3
Name	example.com	example.com	example.com
Type	A	A	A
Policy	Multivalue Answer	Multivalue Answer	Multivalue Answer
Value	IP-1	IP-2	IP-3
Health Check	Check-1	Check-2	Check-3

Test — multiple IPs returned:

```
dig yourdomain.com
# Should show up to 8 IPs in the answer section
```

If one server goes down, its IP is automatically excluded from responses.

Summary & Cheat Sheet

DNS Record Quick Reference

Record	Maps	Example
A	Domain → IPv4	<code>site.com</code> → <code>1.2.3.4</code>
AAAA	Domain → IPv6	<code>site.com</code> → <code>2001:db8::1</code>
CNAME	Domain → Domain	<code>www.site.com</code> → <code>site.com</code>
MX	Domain → Mail server	<code>site.com</code> → <code>mail.google.com</code>
TXT	Domain → Text	SPF, DKIM, verification
NS	Domain → Name servers	Auto-managed by Route 53
PTR	IP → Domain	Reverse lookup
SRV	Service → Host:Port	VoIP, gaming
Alias	Domain → AWS resource	<code>site.com</code> → <code>ALB endpoint</code>

Routing Policy Quick Picker

Need basic single server?	→ Simple Routing
Rolling out a new feature?	→ Weighted Routing
Global app, speed matters?	→ Latency Routing
Need a backup if server fails?	→ Failover Routing
Different content per country?	→ Geolocation Routing
Custom traffic shaping?	→ Geoproximity Routing
Multiple servers, basic spread?	→ Multi-Value Routing
Route by network/ISP?	→ IP-Based Routing

Key Concepts to Remember

- ✓ Alias records are FREE for AWS targets
- ✓ CNAME cannot be used at root domain – use Alias instead
- ✓ Health checks are REQUIRED for Failover routing
- ✓ TTL affects how quickly DNS changes take effect
- ✓ Private hosted zones only work inside VPCs
- ✓ Route 53 health checkers are public – cannot reach private IPs
- ✓ Latency routing ≠ Geolocation routing
- ✓ Multi-Value is NOT a replacement for a Load Balancer
- ✓ DNS resolves names – it does not route actual traffic packets
- ✓ Always add a Default record in Geolocation routing

Notes compiled from hands-on labs and AWS documentation study sessions.

ACM — SSL/TLS Certificates

💡 What is ACM?

AWS Certificate Manager (ACM) is a managed service that handles the entire lifecycle of SSL/TLS certificates for your AWS resources — requesting, validating, deploying, and automatically renewing them before they expire.

Without ACM, you would need to manually purchase certificates from a Certificate Authority, install them on each server, track expiry dates, and renew them every year. ACM eliminates all of that overhead entirely.

Traditional Certificate Process:

Buy cert from CA (~\$100/year) →
Install manually on each server →
Track expiry yourself →
Renew every year →
Update config on renewal →

With ACM:

Free with AWS services
Attach to ALB in 2 clicks
Auto-renews before expiry
Never expires if attached
Zero-downtime renewal

The most important thing to understand: **ACM certificates are completely free** when used with integrated AWS services like Application Load Balancers, CloudFront distributions, and API Gateway. You only pay for the underlying AWS service, never for the certificate itself.

🔑 Key Concepts

Term	Plain English Explanation
SSL/TLS	The encryption protocol that makes HTTPS work — scrambles data between browser and server so nobody can intercept it
TLS Termination	The ALB decrypts incoming HTTPS traffic and forwards plain HTTP to your backend instances — instances don't need to handle encryption
End-to-End Encryption	HTTPS all the way from browser → ALB → backend instances — traffic is never decrypted in transit
DNS Validation	Proving domain ownership by adding a specific CNAME record to your DNS — recommended method
Email Validation	Proving domain ownership by clicking a link sent to admin emails like admin@yourdomain.com
Wildcard Certificate	A single certificate that covers a domain and ALL its subdomains (*.example.com covers www, api, mail, shop, etc.)
Certificate ARN	The unique identifier AWS assigns to your certificate — used when attaching it to ALB or CloudFront

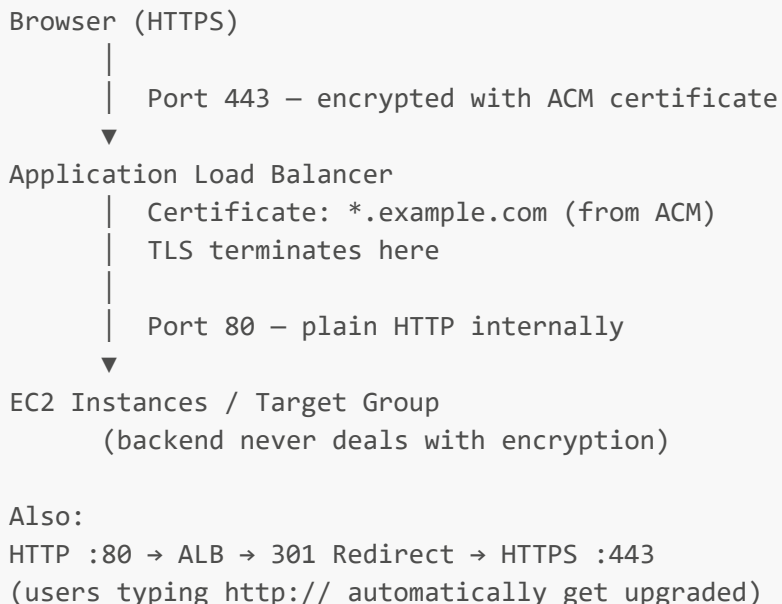
🏠 Why Use ACM?

4 Core Reasons:

1. 🔒 HTTPS is Required Today
Browsers show "Not Secure" on HTTP sites
Google penalizes non-HTTPS sites in search rankings
Users don't trust sites without the padlock icon
2. 💰 Completely Free
No annual certificate fees
No renewal costs
Only pay for the AWS service it's attached to
3. ♻️ Auto-Renewal
ACM renews certificates automatically before expiry
No reminders, no manual work, no downtime from expired certs
4. 🧩 Native AWS Integration
Works directly with ALB, CloudFront, API Gateway
Route 53 + ACM = one-click DNS validation
No manual installation on servers

🚀 Hands-On: Request, Validate, and Attach a Certificate

What We're Building



📋 Step 1 — Request a Certificate

Navigate to ACM:

- AWS Console → Search "**Certificate Manager**" → Open ACM
- Make sure you are in the **correct region** — certificates are region-specific

⚠ If you are attaching the certificate to CloudFront, you must request it in **us-east-1 (N. Virginia)** regardless of where your other resources are. For ALB, request it in the same region as your ALB.

- Click "**Request a certificate**"
- Select "**Request a public certificate**" → Click "**Next**"

Add Your Domain Names:

Domain	Type	Covers
example.com	Exact match	Root domain only
*.example.com	Wildcard	All subdomains (www, api, shop, mail, etc.)

Best practice: always request both **example.com** AND ***.example.com** together in a single certificate. This covers your root domain and every subdomain with one cert.

Validation Method:

Method	How It Works	When to Use
DNS Validation ✓	Add a CNAME record to your DNS	Recommended — works with Route 53 automatically
Email Validation	Click link in email to admin@yourdomain.com	Use only if you don't control the DNS

- Select "**DNS validation**" → Click "**Request**"

📋 Step 2 — Validate Domain Ownership

After requesting, ACM gives you a CNAME record that proves you own the domain.

What the validation record looks like:

ACM gives you this CNAME to add to DNS:

Name: `_a79865eb4cd1a6ab56f5b50a..example.com`
Value: `_1234567890abcdef.acm-validations.aws.`

This record proves to AWS that you control example.com

Option A — Domain is in Route 53 (Easiest):

- On the certificate details page → expand **Domains** section
- Click "**Create records in Route 53**" button
- ACM automatically adds the CNAME records to your hosted zone

- Click **"Create records"** → Done 

Option B — Domain is with GoDaddy or Namecheap:

Go to your registrar's DNS management panel and manually add:

Field	Value
Record Type	CNAME
Name	<code>_a79865eb4cd1a6ab56f5b50a</code> (the part before your domain)
Value	<code>_1234567890abcdef.acm-validations.aws.</code>
TTL	300

Validation Timeline:

Record added → DNS propagates globally → ACM detects it → Certificate issued

Typical wait time: 5 to 30 minutes

Maximum wait time: 72 hours (DNS propagation across globe)

Status changes:

Pending validation → (wait) → Issued 

- Monitor status in **ACM** → **Certificates** — refresh the page every few minutes
- Once status shows **"Issued"** — your certificate is ready to use

Step 3 — Attach Certificate to ALB

Add HTTPS Listener:

- EC2 → Load Balancers → Select your ALB
- Click the **"Listeners and rules"** tab
- Click **"Add listener"**

Field	Value
Protocol	HTTPS
Port	443
Default action	Forward to → select your target group
Default SSL/TLS certificate	Select your ACM certificate from dropdown

- Click **"Add"** 

Redirect HTTP to HTTPS (Best Practice):

Users might type `http://example.com` directly. You want them automatically upgraded to HTTPS.

- In the Listeners tab → find the **HTTP : 80** listener
- Click on it → **Edit listener**
- Change the default action to: **Redirect to URL**

Field	Value
Protocol	HTTPS
Port	443
Status code	301 — Moved Permanently

- Click **"Save changes"** 

Result of 301 redirect:

```
User types:      http://example.com
Browser sends:   GET http://example.com
ALB responds:    301 → https://example.com
Browser follows: GET https://example.com 
Browser saves:   Remembers to always use HTTPS next time
```

TLS Termination vs End-to-End Encryption

These are two different architectural approaches to where HTTPS encryption lives in your stack.

Option A — TLS Termination at ALB (Most Common)

```
Browser —[HTTPS encrypted]→ ALB —[HTTP plain]→ EC2 Instances
                        ↑
                Decryption happens here
                ACM certificate lives here
```

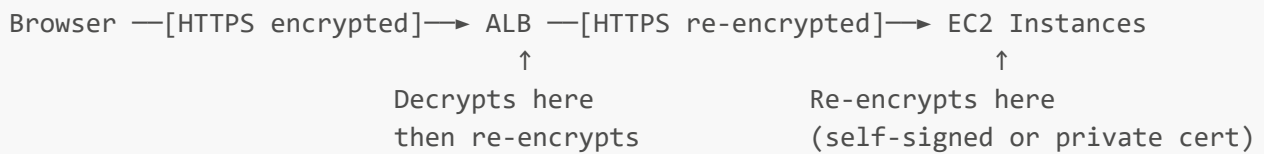
Pros:

-  Simpler backend – EC2 instances don't handle SSL
-  Lower CPU on backend instances (no encryption overhead)
-  ACM handles everything – no certs to install on servers
-  ALB can inspect HTTP headers for routing decisions

Cons:

-  Traffic between ALB and EC2 is unencrypted
-  (This is inside AWS's private network – generally acceptable)

Option B — End-to-End Encryption

**Pros:**

- ✓ Data encrypted at every hop – even inside AWS network
- ✓ Required for strict compliance (PCI-DSS, HIPAA)

Cons:

- ⚠ Must install certificates on every EC2 instance
- ⚠ Higher CPU usage on backend servers
- ⚠ More complex setup and maintenance
- ⚠ Target group protocol must be set to HTTPS

Which should you use?

Use TLS Termination (Option A) when:

- Standard web applications
- Internal backend traffic stays within AWS VPC
- You want simplicity and lower operational overhead

Use End-to-End Encryption (Option B) when:

- Strict compliance requirements (healthcare, payments)
- Regulatory mandates require encrypted data at every layer
- Your security policy requires no plaintext anywhere

ACM Quick Reference

Region Rules:

- ALB certificate → Same region as ALB
- CloudFront certificate → MUST be us-east-1 (N. Virginia)

✓ Always Request Both:

- example.com → Covers root domain
- *.example.com → Covers all subdomains

Validation Methods:

- Route 53 + ACM → One-click automatic (recommended)
- Other registrar → Manual CNAME record addition

Wait Times:

- DNS validation → 5 to 30 minutes typically
- Certificate status → Pending validation → Issued

Cost:

- Public certificate → Free with ALB, CloudFront, API Gateway

Private certificate → \$400/month per CA



HTTP to HTTPS Redirect:

HTTP :80 listener → Action: Redirect → HTTPS :443 → 301

Always do this – never leave HTTP open without redirect
